Week 11 — Threads and Concurrency

Format: seven daily sections collected into one weekly module

Suggested workload: approximately 21 focused hours, adjustable to mastery

> A thread is a private register set and a private stack in a shared address space. Everything else this week studies — races, mutexes, condition waits, atomics, deadlocks, cache lines, and debuggers — is a consequence of that split.

The lab is the canonical Linux system-programming project. This week does not copy or patch it. Each day's wrapper resolves that project, builds through its Makefile, and prints what the run is allowed to mean. Kernel excerpts in Days 4–6 were read from tree `c9b2385d6793`, whose `Makefile` says Linux 5.10.134. The host that runs the lab may be a different kernel; `uname` in the reports is that host.

Where Linux 0.12 helps—and where it stops

Start with the sleep/wakeup and task-switch map in the shared Linux 0.12 → Linux 5.10 reference. It makes the reason for task state, sleeping, wakeup, and scheduling visible in one small kernel. Linux 0.12 has no modern pthread/`clone` sharing model, futex subsystem, SMP memory ordering, C11 atomics, RCU, or per-CPU runqueues. Those mechanisms therefore stay grounded in this week's 5.10 excerpts and the C/POSIX contracts.

Why these seven days belong together

Shared memory is one virtual address space, translated by one set of page tables, backed by cache lines the coherence protocol moves as wholes. Each thread still has its own registers, its own stack, and its own place in the scheduler. The C abstract machine then decides which of those concurrent accesses have a meaning at all.

```
pthread / clone
  shared mappings, private registers, private stacks, TLS
        |
        v
C abstract machine
  conflicting non-atomic accesses with no happens-before are undefined
        |
        v
mutex
  a userspace atomic on the fast path, a futex when the task must sleep
        |
        v
condition variable
  check the predicate under the mutex, release and sleep as one step,
  recheck on every wake
        |
        v
C11 atomics and memory orders
  modification order, release/acquire, seq_cst
  compiler reorderings, then CPU reorderings (x86-64 TSO, AArch64, RISC-V)
        |
        v
liveness and the cache
  wait-for cycles; false sharing on one coherence line
        |
        v
tools and three systems
  GDB, Memcheck, ASan/UBSan, TSan
  ToyOS tasks, a teaching-OS sleep, Linux tasks plus the C model
```

Daily roadmap

Day 1 — Threads, processes, `clone`, stacks, and TLS

Day 1 separates the POSIX thread from a new process: shared address space, private registers, stack, signal mask, `errno`, and TLS. Linux libc creates that shape with a `clone`-family call, and the join is both a lifetime edge and a happens-before edge. Wrapper: challenges/day-001-thread-model.sh.

Day 2 — Data races and the C abstract machine

Day 2 defines a data race as conflicting actions, at least one of them non-atomic, with no happens-before between them. A plausible `counter++` total is still undefined behavior. The intentional race runs only through the Day 2 and Day 7 wrappers. Wrapper: challenges/day-002-data-races.sh.

Day 3 — Mutexes, ownership, lock order, and the futex boundary

Day 3 ties a mutex to a named invariant and to the unlock/lock happens-before edge. Heap ownership moves at the join. Two locks need one global order. An uncontended lock can stay in userspace; contention may enter the kernel through `futex`. Wrapper: challenges/day-003-mutex-ownership.sh.

Day 4 — Condition variables and bounded queues

Day 4 starts from the gap a mutex cannot close. The waiter must check `count` while holding the mutex, then release the mutex and become a published sleeper as one operation, then re-acquire and check again. A signal that finds an empty wait list stores nothing. Spurious returns and stolen wakeups are why the lab's `put` and `get` use `while`. Empty and full are the two predicates of an eight-slot ring. glibc implements `pthread_cond_t` in userspace and blocks with `futex`; the lesson quotes `wait_event`, `set_current_state`, and `futex_wait` from tree `c9b2385d6793` only. Wrapper: challenges/day-004-condition-queue.sh.

Day 5 — C11 atomics and memory ordering

Day 5 separates modification order from synchronization. Relaxed fetch-adds on one counter have a single order, so 800000 increments finish at 800000 after the joins. Release and acquire publish a payload. Seq_cst adds one total order, which is why both-zero is outside a seq_cst store-buffering litmus and inside the relaxed litmus the lab runs. The compiler reorders under the C model; the CPU reorders under the ISA. In the excerpt tree, x86-64 `__smp_store_release` is a compiler barrier plus a plain store, AArch64 emits `stlr`/`ldar`, and RISC-V emits `fence`. A litmus count on x86-64 does not prove the C program. Wrapper: challenges/day-005-atomics-memory-order.sh.

Day 6 — Liveness, false sharing, and scalability

Day 6 draws deadlock as a wait-for cycle and breaks it with one global lock order. The detector forces the cycle, waits with `pthread_mutex_timedlock`, and exits; the process does not hang. False sharing is Week 2's cache line: two cores writing different atomic objects on one line bounce that line through the coherence protocol (the MESI picture of exclusive ownership). The kernel header `include/linux/cache.h` in the excerpt tree defines `____cacheline_aligned_in_smp` for the same reason. Adding cores helps private lines and adds contenders on a shared written line. The three timing samples are one machine, one compiler, one layout. Wrapper: challenges/day-006-liveness-scalability.sh.

Day 7 — Debuggers, sanitizers, and three systems

Day 7 puts GDB, Valgrind Memcheck, ASan/UBSan, and ThreadSanitizer on different layers: architectural state at a stop, heap definedness, compiler-inserted shadow checks, and conflicting accesses missing happens-before. ToyOS at its scheduler milestone runs two ring-0 tasks, cooperatively and from the timer, with `TASK_BLOCKED` meaning finished and with no C memory model. A small teaching OS adds `sleep`/`wakeup` on a channel, which you quote only after pinning a revision. Linux adds tasks, futex, and the C model the lab is written in. Wrapper: challenges/day-007-debug-report.sh.

Safety and evidence

The intentional data race has undefined behavior. Run it only through the Day 2 or Day 7 wrapper. A ThreadSanitizer build can fail to link or to start when `libtsan` is missing or the runtime cannot map itself. A skip is not a pass. A finished TSan run that does not report the race is not proof the accesses were ordered; the wrapper records that outcome as a skip.

The deadlock exercise uses a timed lock, then unlocks, joins, and exits. It demonstrates a wait-for cycle without leaving a hung process.

False-sharing timings and the relaxed litmus are measurements. They are not correctness tests. The deterministic suite checks that those programs print their caveat lines. An x86-64 `both_zero` count, including zero, describes that run. Portable meaning stays with the C memory model, the cited ISA manuals, and the compiler output you actually have.

Completion checkpoint

Finishing the wrappers does not complete mastery. Answer from the source and the logs, with hashes and versions, and leave a question blank when the run did not speak to it:

1. Which state is process-wide, which is per-thread, and who owns each object?
2. Which operations conflict, and what happens-before edge orders them?
3. Why can `counter++` fail in the abstract machine even when a run prints the expected total?
4. What invariant does each mutex protect, and what global lock order holds?
5. Why does a condition wait recheck its predicate while holding the mutex?
6. Which memory order matches the protocol, and which kernel mapping (`barrier` plus store, `stlr`/`ldar`, or `fence`) did you actually read?
7. Which claim came from an assertion, a timer, a debugger, a sanitizer, a compiler listing, an ISA manual, or the excerpt tree?
8. What is still open because of scheduling, optimization, architecture, a missing `libtsan`, or a two-thread sample on one machine?

Do not auto-fill mastery, confidence, hours, or completion. The quizzes and week-11-material.pdf sit beside this overview and do not grade the report.

Day 1 — Threads, Processes, `clone`, Stacks, and TLS

Target time: approximately 3 hours

> `pthread_create` returns, and a second stream of instructions is running in the same process. Which objects did the kernel share, which did it allocate, and which register now names this thread's private data?

Kernel excerpts below are from tree `c9b2385d6793` (`Makefile` `VERSION` 5, `PATCHLEVEL` 10, `SUBLEVEL` 134). Each excerpt names its path and function.

Week 10 built one task: an address space, a descriptor table, signal dispositions, and a single set of registers. Today a second task joins that same address space. The mechanism is the `clone` syscall you already met as the implementation behind `fork`, with a different flag set and a new stack pointer.

---

1. Why a second instruction stream can share an address space

A function call has one instruction pointer. The callee borrows `%rsp` and returns. A second thread needs its own instruction pointer at the same moment, so it needs its own saved registers: one core holds one architectural register file. It needs its own user stack, because the System V AMD64 stack grows down from `%rsp` and two streams would overwrite each other's frames. It needs its own kernel stack, because a syscall or timer interrupt lands on the current task's kernel stack, the same privilege transition Week 8 already traced.

The program text, the heap, and the open files already live in the address space and the kernel objects from Week 10. Two tasks that hold the same `mm_struct` resolve the same virtual address to the same object. That is why the worker in `thread_lab.c` can receive `&token`, an automatic `int` in `mode_basic`, and return that pointer. The creator's stack page is mapped in the shared address space for as long as that frame is alive, which is why `mode_basic` joins before it returns.

```
one mm_struct
┌──────────────────────────────────────────────────────┐
│  text, rodata, data, heap, libc, vDSO                │
│                                                      │
│  user stack of task A          user stack of task B  │
│  (creator %rsp)                (newsp from clone)    │
│  TLS block A                   TLS block B           │
└──────────────────────────────────────────────────────┘
        │                                │
        ▼                                ▼
   task_struct A                    task_struct B
   kernel stack A                   kernel stack B
   pt_regs / user registers         pt_regs / user registers
   blocked signal mask              blocked signal mask
   FS base → TLS block A            FS base → TLS block B
```

Per task: general-purpose registers, kernel stack, user stack, TLS base, blocked-signal mask. Process-wide for a normal pthread: the `mm`, the file descriptor table, and the signal dispositions. The thread-group id, the number `getpid` returns, is shared as well.

---

2. Tasks you have already built

Week 7's kernel tasks are two ring-0 streams in one address space. Each has a private kernel stack and a saved context. The page tables stay shared. The separation is the stack and the registers.

Week 8 gives ToyOS one user address space: its own page-table root, a user stack, and a kernel stack for the privilege transition. That shape is a separate address space plus a kernel stack.

Week 10's `fork` is the Linux form of a new address space. `SYSCALL_DEFINE0(fork)` calls `kernel_clone` with `exit_signal = SIGCHLD` and the rest of `kernel_clone_args` zeroed. The child gets its own `mm` (copy-on-write) and a copy of the parent's user `%rsp`: the same virtual address in a different address space.

`pthread_create` is a libc wrapper around `clone` (or `clone3`, which reaches the same `kernel_clone`). The flags share the `mm`, the file table, the signal handlers, and the thread-group id, and the wrapper passes a new user stack and a TLS pointer.

---

3. How `clone` reaches `kernel_clone`

On x86-64 this tree selects the parameter order in the `#else` branch of `kernel/fork.c`. `arch/x86/Kconfig` selects `CLONE_BACKWARDS` only for `X86_32`, so a 64-bit kernel compiles the signature below. The body that fills `struct kernel_clone_args` is shared with the other orders.

```
/* kernel/fork.c :: SYSCALL_DEFINE5(clone) :: tree c9b2385d6793
 * lines 2697–2714, the order x86-64 compiles
 */
SYSCALL_DEFINE5(clone, unsigned long, clone_flags, unsigned long, newsp,
         int __user *, parent_tidptr,
         int __user *, child_tidptr,
         unsigned long, tls)
#endif
{
    struct kernel_clone_args args = {
        .flags      = (lower_32_bits(clone_flags) & ~CSIGNAL),
        .pidfd      = parent_tidptr,
        .child_tid  = child_tidptr,
        .parent_tid = parent_tidptr,
        .exit_signal    = (lower_32_bits(clone_flags) & CSIGNAL),
        .stack      = newsp,
        .tls        = tls,
    };

    return kernel_clone(&args);
}
```

The x86-64 `syscall` instruction delivers arguments in `%rdi`, `%rsi`, `%rdx`, `%r10`, and `%r8`: flags, new user stack, parent tid pointer, child tid pointer, and TLS. The low 8 bits (`CSIGNAL`) become `exit_signal` and are removed from `flags`. `fork` sets that byte to `SIGCHLD`. A thread-creating caller leaves it clear and puts the sharing flags in the rest of the word.

`kernel_clone` is the common path for `fork`, `vfork`, `clone`, and `clone3`. The part that builds the task and lets it run is:

```
/* kernel/fork.c :: kernel_clone :: tree c9b2385d6793
 * lines 2589–2605
 */
    p = copy_process(NULL, trace, NUMA_NO_NODE, args);
    add_latent_entropy();

    if (IS_ERR(p))
        return PTR_ERR(p);

    /*
     * Do this prior waking up the new thread - the thread pointer
     * might get invalid after that point, if the thread exits quickly.
     */
    trace_sched_process_fork(current, p);

    pid = get_task_pid(p, PIDTYPE_PID);
    nr = pid_vnr(pid);

    if (clone_flags & CLONE_PARENT_SETTID)
        put_user(nr, args->parent_tid);
```

`copy_process` returns a task that is fully built and still off the run queue. After the vfork and LRU bookkeeping that follows this excerpt, line 2620 calls `wake_up_new_task`, which makes the child schedulable. The parent syscall returns `nr` in `%rax`. The child's saved `%rax` is 0, so both tasks resume the libc wrapper and branch on `%rax`: the parent continues `pthread_create`, and the child enters the thread start routine.

---

4. `copy_process` copies or shares one resource at a time

Before any of those helpers run, `dup_task_struct` (`kernel/fork.c` line 907) allocates a new `task_struct` and a new kernel stack (`alloc_thread_stack_node` at line 920). `arch_dup_task_struct` copies the parent with `*dst = *src`, so the child starts life with the parent's pointers. The copy helpers then either bump a reference count and keep the pointer, or allocate a replacement. The kernel stack pointer is rewritten after the copy (`tsk->stack = stack` at line 936), which is why the child has a private kernel stack even though the struct was memcpy'd.

`include/linux/sched.h` places the process-wide pointers and the per-task mask side by side (lines 1129–1132): `signal`, `sighand`, and `sigset_t blocked`. The blocked mask is a member of the task, so the struct copy gives the child its own mask, initialized to the parent's, and a later `pthread_sigmask` changes one task.

The resource walk itself:

```
/* kernel/fork.c :: copy_process :: tree c9b2385d6793
 * lines 2176–2197
 */
    retval = copy_files(clone_flags, p);
    if (retval)
        goto bad_fork_cleanup_semundo;
    retval = copy_fs(clone_flags, p);
    if (retval)
        goto bad_fork_cleanup_files;
    retval = copy_sighand(clone_flags, p);
    if (retval)
        goto bad_fork_cleanup_fs;
    retval = copy_signal(clone_flags, p);
    if (retval)
        goto bad_fork_cleanup_sighand;
    retval = copy_mm(clone_flags, p);
    if (retval)
        goto bad_fork_cleanup_signal;
    retval = copy_namespaces(clone_flags, p);
    if (retval)
        goto bad_fork_cleanup_mm;
    retval = copy_io(clone_flags, p);
    if (retval)
        goto bad_fork_cleanup_namespaces;
    retval = copy_thread(clone_flags, args->stack, args->stack_size, p, args->tls);
```

`args->stack` is the user `newsp` from the syscall. `args->tls` is the TLS argument. Both reach the architecture code only here. `copy_fs` is the same pattern for the current directory and umask (`CLONE_FS`); pthread libc often shares it too. The four helpers the rest of this lesson reads are `copy_mm`, `copy_files`, `copy_sighand`, and `copy_signal`.

The kernel refuses a thread group that fails to share the address space. The check is earlier in `copy_process`:

```
/* kernel/fork.c :: copy_process :: tree c9b2385d6793
 * lines 1950–1963
 */
    /*
     * Thread groups must share signals as well, and detached threads
     * can only be started up within the thread group.
     */
    if ((clone_flags & CLONE_THREAD) && !(clone_flags & CLONE_SIGHAND))
        return ERR_PTR(-EINVAL);

    /*
     * Shared signal handlers imply shared VM. By way of the above,
     * thread groups also imply shared VM. Blocking this case allows
     * for various simplifications in other code.
     */
    if ((clone_flags & CLONE_SIGHAND) && !(clone_flags & CLONE_VM))
        return ERR_PTR(-EINVAL);
```

`CLONE_THREAD` requires `CLONE_SIGHAND`, and `CLONE_SIGHAND` requires `CLONE_VM`. A thread group shares both the dispositions and the address space. The kernel enforces that before the new task runs.

---

5. `copy_mm`: `CLONE_VM` shares the address space

`copy_mm` starts at line 1456 by clearing fault counters and setting `tsk->mm` and `tsk->active_mm` to `NULL`. The decision is then:

```
/* kernel/fork.c :: copy_mm :: tree c9b2385d6793
 * lines 1471–1497
 */
    /*
     * Are we cloning a kernel thread?
     *
     * We need to steal a active VM for that..
     */
    oldmm = current->mm;
    if (!oldmm)
        return 0;

    /* initialize the new vmacache entries */
    vmacache_flush(tsk);

    if (clone_flags & CLONE_VM) {
        mmget(oldmm);
        mm = oldmm;
        goto good_mm;
    }

    retval = -ENOMEM;
    mm = dup_mm(tsk, current->mm);
    if (!mm)
        goto fail_nomem;

good_mm:
    tsk->mm = mm;
    tsk->active_mm = mm;
    return 0;
```

A kernel thread has `current->mm == NULL`, and `copy_mm` returns with the child's `mm` still `NULL`. That is the Linux shape closest to a Week 7 ToyOS task: execution state with a shared (or absent) user address space, and the private state living in the stack.

`fork` leaves `CLONE_VM` clear, so `dup_mm` builds a new `mm_struct`. Virtual addresses survive and the physical pages stay copy-on-write until a write. A pthread sets `CLONE_VM`. `mmget` takes another reference on the same `mm_struct`, and both tasks store that pointer. A store through a pointer in the child is a store to the parent's object. Both cores can cache the line; Week 2's coherence protocol moves exclusive ownership when one of them writes. Day 2 asks what C says about that pair of accesses.

The main thread's stack mapping was created at `execve`, as Week 3 described. Further thread stacks are anonymous mappings libc creates in this shared `mm` before `clone`. `copy_mm` only publishes the `mm` pointer.

---

6. `copy_files`: `CLONE_FILES` shares the descriptor table

```
/* kernel/fork.c :: copy_files :: tree c9b2385d6793
 * lines 1523–1548
 */
static int copy_files(unsigned long clone_flags, struct task_struct *tsk)
{
    struct files_struct *oldf, *newf;
    int error = 0;

    /*
     * A background process may not have any files ...
     */
    oldf = current->files;
    if (!oldf)
        goto out;

    if (clone_flags & CLONE_FILES) {
        atomic_inc(&oldf->count);
        goto out;
    }

    newf = dup_fd(oldf, NR_OPEN_MAX, &error);
    if (!newf)
        goto out;

    tsk->files = newf;
    error = 0;
out:
    return error;
}
```

With `CLONE_FILES`, the function increments the reference count and returns, leaving the pointer `dup_task_struct` already copied. Both tasks index the same `files_struct`. `open`, `close`, and `dup` in one thread change the table the other thread sees. Descriptor 3 in either thread is the same slot.

`fork` leaves the flag clear, so `dup_fd` allocates a new table whose entries still point at the same open-file descriptions. Week 10 separated those two layers. A pthread shares the table itself. The `files_struct` is kernel memory reached through the task pointer; a heap `int` is shared because both tasks walk the same page tables.

---

7. Dispositions are shared; the thread-group id is shared

Signal dispositions live in `sighand->action[]` (`include/linux/sched/signal.h`, `struct sighand_struct`). `copy_sighand`:

```
/* kernel/fork.c :: copy_sighand :: tree c9b2385d6793
 * lines 1576–1592
 */
static int copy_sighand(unsigned long clone_flags, struct task_struct *tsk)
{
    struct sighand_struct *sig;

    if (clone_flags & CLONE_SIGHAND) {
        refcount_inc(&current->sighand->count);
        return 0;
    }
    sig = kmem_cache_alloc(sighand_cachep, GFP_KERNEL);
    RCU_INIT_POINTER(tsk->sighand, sig);
    if (!sig)
        return -ENOMEM;

    refcount_set(&sig->count, 1);
    spin_lock_irq(&current->sighand->siglock);
    memcpy(sig->action, current->sighand->action, sizeof(sig->action));
    spin_unlock_irq(&current->sighand->siglock);
```

`CLONE_SIGHAND` bumps the count and keeps the pointer `dup_task_struct` copied. `sigaction` in one thread installs a handler every thread will run. The other branch allocates a new table and memcpy's the current actions; `fork` takes that branch, which is why a child's later `sigaction` leaves the parent alone. `CLONE_CLEAR_SIGHAND`, just below this excerpt, resets handlers to the default as a further option.

The thread group itself is `struct signal_struct`, pointed to by `task->signal`. `copy_signal`:

```
/* kernel/fork.c :: copy_signal :: tree c9b2385d6793
 * lines 1625–1647
 */
static int copy_signal(unsigned long clone_flags, struct task_struct *tsk)
{
    struct signal_struct *sig;

    if (clone_flags & CLONE_THREAD)
        return 0;

    sig = kmem_cache_zalloc(signal_cachep, GFP_KERNEL);
    tsk->signal = sig;
    if (!sig)
        return -ENOMEM;

    sig->nr_threads = 1;
    atomic_set(&sig->live, 1);
    refcount_set(&sig->sigcnt, 1);

    /* list_add(thread_node, thread_head) without INIT_LIST_HEAD() */
    sig->thread_head = (struct list_head)LIST_HEAD_INIT(tsk->thread_node);
    tsk->thread_node = (struct list_head)LIST_HEAD_INIT(sig->thread_head);

    init_waitqueue_head(&sig->wait_chldexit);
    sig->curr_target = tsk;
    init_sigpending(&sig->shared_pending);
```

`CLONE_THREAD` returns immediately. The child's `signal` pointer stays the one copied from the parent, `nr_threads` is not reset to 1, and the child joins the existing group. The other branch allocates a fresh group whose thread list contains only the new task. That is `fork`.

The id userspace calls the pid is assigned a few hundred lines later:

```
/* kernel/fork.c :: copy_process :: tree c9b2385d6793
 * lines 2260–2272
 */
    /* ok, now we should be set up.. */
    p->pid = pid_nr(pid);
    if (clone_flags & CLONE_THREAD) {
        p->group_leader = current->group_leader;
        p->tgid = current->tgid;
    } else {
        p->group_leader = p;
        p->tgid = p->pid;
    }

    p->nr_dirtied = 0;
    p->nr_dirtied_pause = 128 >> (PAGE_SHIFT - 10);
    p->dirty_paused_when = 0;
```

Each task has its own `pid` (the tid). `CLONE_THREAD` copies `tgid` and `group_leader` from the caller. `getpid` reports `tgid`. `gettid` reports `pid`. `/proc/self` names the group; `/proc/thread-self` names the task. The `nr_dirtied` fields are accounting reset on the same path; the identity rule is the `tgid` branch.

The signal picture for a pthread is:

```
shared signal_struct     tgid, group leader, shared pending, rlimits
shared sighand_struct    disposition table (SIG_DFL / SIG_IGN / handler)
per task                 blocked, real_blocked, private pending
```

`sigaction` updates the shared `action[]`. `pthread_sigmask` updates one task's `blocked`.

---

8. Private registers, two stacks, and a TLS base

The architecture half of creation is `arch/x86/kernel/process.c`, `copy_thread`. The new kernel stack is framed so the first time the scheduler switches to this task it resumes at `ret_from_fork`:

```
/* arch/x86/kernel/process.c :: copy_thread :: tree c9b2385d6793
 * lines 146–163
 */
    childregs = task_pt_regs(p);
    fork_frame = container_of(childregs, struct fork_frame, regs);
    frame = &fork_frame->frame;

    frame->bp = encode_frame_pointer(childregs);
    frame->ret_addr = (unsigned long) ret_from_fork;
    p->thread.sp = (unsigned long) fork_frame;
    p->thread.io_bitmap = NULL;
    clear_tsk_thread_flag(p, TIF_IO_BITMAP);
    p->thread.iopl_warn = 0;
    memset(p->thread.ptrace_bps, 0, sizeof(p->thread.ptrace_bps));

#ifdef CONFIG_X86_64
    current_save_fsgs();
    p->thread.fsindex = current->thread.fsindex;
    p->thread.fsbase = current->thread.fsbase;
    p->thread.gsindex = current->thread.gsindex;
    p->thread.gsbase = current->thread.gsbase;
```

`p->thread.sp` points at a frame on the new kernel stack, and `ret_from_fork` is the address that frame returns to. `childregs` is the saved user register block on that stack. The parent's `thread.sp` stays where it was. On x86-64 the same function snapshots the parent's FS and GS bases; FS is the System V AMD64 TLS base, and `CLONE_SETTLS` replaces it for a pthread.

The user register block is then a copy of the parent's syscall frame, with `%rax` and `%rsp` edited:

```
/* arch/x86/kernel/process.c :: copy_thread :: tree c9b2385d6793
 * lines 194–223
 */
    frame->bx = 0;
    *childregs = *current_pt_regs();
    childregs->ax = 0;
    if (sp)
        childregs->sp = sp;

#ifdef CONFIG_X86_32
    task_user_gs(p) = get_user_gs(current_pt_regs());
#endif

    if (unlikely(p->flags & PF_IO_WORKER)) {
        /*
         * An IO thread is a user space thread, but it doesn't
         * return to ret_after_fork().
         *
         * In order to indicate that to tools like gdb,
         * we reset the stack and instruction pointers.
         *
         * It does the same kernel frame setup to return to a kernel
         * function that a kernel thread does.
         */
        childregs->sp = 0;
        childregs->ip = 0;
        kthread_frame_init(frame, sp, arg);
        return 0;
    }

    /* Set a new TLS for the child thread? */
    if (clone_flags & CLONE_SETTLS)
        ret = set_new_tls(p, tls);
```

`childregs->ax = 0` is the child's `%rax`. Non-zero `sp` (the syscall's `newsp`) becomes the child's user `%rsp`, so the child resumes on the stack libc prepared. Zero `sp`, as in `fork`, keeps the copied user `%rsp` inside the child's new `mm`. `PF_IO_WORKER` is a kernel-internal user task that returns through `kthread_frame_init`; a pthread falls through to `CLONE_SETTLS`.

`set_new_tls` (lines 129–136) calls `do_set_thread_area_64(p, ARCH_SET_FS, tls)`, which the header macro (`arch/x86/include/asm/ptrace.h` line 386) turns into `do_arch_prctl_64`. Libc has already written a startup frame on `newsp`. After `ret_from_fork` the child is in that stub, sees `%rax == 0`, and calls `return_id` or `consume_owned`.

---

9. Architecture lens: which register holds the thread pointer

The TLS block is ordinary memory in the shared `mm`. The pointer to it has to be a value context-switch saves with the task, because the thread must be able to find its own block before it has loaded any other address. On x86-64 that value is the FS base:

```
/* arch/x86/kernel/process_64.c :: do_arch_prctl_64 :: tree c9b2385d6793
 * lines 785–813, case ARCH_SET_FS
 */
    case ARCH_SET_FS: {
        /*
         * Not strictly needed for %fs, but do it for symmetry
         * with %gs
         */
        if (unlikely(arg2 >= TASK_SIZE_MAX))
            return -EPERM;

        preempt_disable();
        /*
         * Set the selector to 0 for the same reason
         * as %gs above.
         */
        if (task == current) {
            loadseg(FS, 0);
            x86_fsbase_write_cpu(arg2);

            /*
             * On non-FSGSBASE systems, save_base_legacy() expects
             * that we also fill in thread.fsbase.
             */
            task->thread.fsbase = arg2;
        } else {
            task->thread.fsindex = 0;
            x86_fsbase_write_task(task, arg2);
        }
        preempt_enable();
        break;
    }
```

A newborn task takes the `else` branch: FS index 0, and the base stored on the task for the next switch to load. User code then reads `%fs:offset`. The System V AMD64 ABI uses that base as the thread pointer and puts the stack canary at `%fs:0x28`. Libc's `errno` is one such TLS object: the kernel returns one code in `%rax`, and each thread's wrapper stores it in that thread's `errno`.

AArch64's user thread pointer is the system register `TPIDR_EL0`. RISC-V's ABI thread pointer is the integer register `tp` (`x4`). This lesson quotes the x86 path that writes the FS base. The arm64 and riscv `copy_thread` functions install their own registers under the same `CLONE_SETTLS` flag.

```
x86-64    FS base, accessed as %fs:offset     quoted above
AArch64   TPIDR_EL0                           ABI register, setup not quoted
RISC-V    tp (x4), accessed as offset(tp)     ABI register, setup not quoted
```

The compiler emits those accesses for `_Thread_local`. The block lives as long as the thread does. A pointer into another thread's TLS is reachable because the `mm` is shared, and the object stays valid while that thread still owns the block.

---

10. Execution trace: `thread_lab basic`

One successful `pthread_create` / `pthread_join` of `return_id(&token)` in `mode_basic`. `token` is an `int` with value 42 in the creator's frame.

```
creator, user mode                          child, still being built
mode_basic:
  token = 42            lives at &token on the creator user stack
  pthread_create(return_id, &token)
    libc allocates a user stack and a TLS block in the shared mm
    libc writes a startup frame on that stack
    syscall clone
      %rdi flags  (CLONE_VM | CLONE_FILES | CLONE_SIGHAND
                   | CLONE_THREAD | CLONE_SETTLS, plus libc extras)
      %rsi newsp  (top of the new user stack)
      %r8  tls    (address of the new TLS block, x86-64 ABI)

kernel, creator's kernel stack
  SYSCALL_DEFINE5(clone) → kernel_clone
  dup_task_struct: new task_struct, new kernel stack
  copy_files:   CLONE_FILES, same files_struct
  copy_sighand: CLONE_SIGHAND, same dispositions
  copy_signal:  CLONE_THREAD, same signal_struct, same tgid
  copy_mm:      CLONE_VM, mmget, same mm
  copy_thread:  kernel frame → ret_from_fork
                childregs->ax = 0
                childregs->sp = newsp
                CLONE_SETTLS → ARCH_SET_FS on the child task
  wake_up_new_task
  parent returns tid in %rax

creator resumes pthread_create              child resumes on newsp
  stores pthread_t                          libc stub sees %rax == 0
  pthread_join                              calls return_id(&token)
                                            return_id returns &token
                                            thread exit
  join copies the returned pointer
  result == &token
  prints pthread_create_join=ok result=42
```

Writes to `token` are sequenced before `pthread_create`, and creation synchronizes with the new thread's start, so the child's read happens after that write. `pthread_join` synchronizes with the child's termination, so the creator's comparison happens after `return_id` returns. The ownership mode uses the same two edges around a heap object: the child writes `value *= 2` only after create, and the parent reads and `free`s only after join. Day 2 names the edge that is absent when two threads update one non-atomic object in between.

---

11. Mastery: Explain · Draw · Observe · Build

Run `../challenges/day-001-thread-model.sh`. It builds `thread_lab`, runs `basic` and `ownership`, and prints the lifetime-relevant source. Predict `pthread_create_join=ok result=42` and `joined_value=42` with `freed_by=parent`. Leave thread ids and scheduling order unpredicted.

Explain

`pthread_create` is libc around `clone` / `kernel_clone`, with flags that share the `mm` (`CLONE_VM`), the file table (`CLONE_FILES`), the signal handlers (`CLONE_SIGHAND`), and the thread-group id (`CLONE_THREAD`), plus a new user stack and TLS (`CLONE_SETTLS`). Per task: registers, kernel stack, user stack, TLS base, `blocked`. Process-wide: `mm`, descriptor table, dispositions, `tgid`. On x86-64, `copy_thread` writes `childregs->sp` and clears `childregs->ax`, and `ARCH_SET_FS` records the TLS base. AArch64 uses `TPIDR_EL0`. RISC-V uses `tp`.

Draw

1. One `mm_struct` containing two user stacks and two TLS blocks, and two `task_struct`s under it with private kernel stacks.
2. The `copy_process` order: files, sighand, signal, mm, then `copy_thread(stack, tls)`.
3. The `basic` trace from `token = 42` through `clone` to the joined pointer comparison. Mark which edge is `pthread_create` and which is `pthread_join`.

Observe

From the challenge output, record:

- the two checked lines (`result=42`, `joined_value=42`);
- `uname` and the compiler version the script prints;
- the source range the script prints, and on each line say whether the object is the shared `mm`, the creator stack, the heap, or a `pthread_t`.

Build

On the source range the script prints, label `token`, the heap `item`, and the `pthread_t` with where the object lives and which intervals of the create/join trace may touch it. Then name the `clone` flag for each shared column: `CLONE_VM`, `CLONE_FILES`, `CLONE_SIGHAND`, `CLONE_THREAD`.

Why? notebook

1. Why does a second thread need a new user stack when it already shares the address space?
2. Why does `fork` leave `newsp` zero while `pthread_create` passes a pointer?
3. Why does `copy_files` under `CLONE_FILES` increment a counter and return without assigning `tsk->files`?
4. Why does the kernel reject `CLONE_THREAD` without `CLONE_SIGHAND`, and `CLONE_SIGHAND` without `CLONE_VM`?
5. Why can `getpid` match in two threads whose `/proc/.../task` ids differ?
6. Why is the blocked-signal mask per task while `sigaction` is shared?
7. Why is the child's `%rax` 0 when the parent receives a tid?
8. Why does x86-64 store the TLS base on the task in the `else` branch of `ARCH_SET_FS`?
9. Why does Week 7's shared page tables plus private kernel stacks line up with `CLONE_VM`, while Week 8's private page-table root lines up with `fork`?

References used selectively

Linux tree `c9b2385d6793`, paths named on each excerpt above.

Next: the shared `mm` makes both threads' loads and stores hit one object. Day 2 asks when the C abstract machine still assigns that program a meaning.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Thread Model

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-001-thread-model.sh
```

- View lab source on GitHub

Day 2 — Data Races and the C Abstract Machine

Target time: approximately 3 hours

> Four threads execute `unsafe_counter++`. The machine is coherent, the compiler is conforming, and the printed total can still be anything — including the total you hoped for. Where did the program lose its meaning?

Day 1 left two tasks in one `mm_struct`. The same virtual address is the same object, and Week 2's caches will move that line between cores until they agree on the bytes. Today's question is older than the cache protocol. It is the question Week 2's abstract machine already posed for a single thread: C constrains observable behavior, and where the program has undefined behavior the implementation has no obligation to preserve the story the source seems to tell.

The lab object is `unsafe_counter` in `thread_lab.c`. Four threads run this loop, `ITERATIONS` equal to 200000:

```
for (int i = 0; i < ITERATIONS; ++i) unsafe_counter++;
```

Kernel wording quoted below is from tree `c9b2385d6793`. The assembly is from this host's GCC 16.2.1 20260819 (Red Hat 16.2.1-2), which is also the compiler that produced `projects/linux-system-programming/build/thread_lab` under the project's `CFLAGS ?= -O2`.

---

1. Why a lost update and a perfect total are one fact

`counter++` on a plain `uint64_t` is a read, a modify, and a write of a non-atomic object. In the abstract machine those are separate accesses. Two threads performing them on the same object, with at least one write, and with no happens-before edge between the accesses, execute a **data race**. A data race is undefined behavior. The standard then imposes no requirements on that execution. The final number, the number of memory operations left in the loop, and the text a later `printf` shows are all outside what the standard still specifies.

Hardware can still finish every instruction. Coherence can still make every core's later load see some agreed value. A particular schedule can still print `800000`, which is `4 * 200000`. That printout is a property of one run of one binary. It is the abstract machine declining to give the run a portable meaning. The challenge script therefore treats the racy total as unconstrained and labels the mode `data_race=undefined_behavior`.

The mutex mode of the same program is a different program. Every increment happens while `counter_lock` is held, unlock synchronizes with a later lock, and the checked result is exactly 800000. Day 3 builds that edge. Today the point is to see the edge's absence in both the language and the machine.

---

2. Conflicting accesses and happens-before

Two accesses **conflict** when they target the same memory location and at least one of them modifies the object. A load conflicts with a store. Two stores conflict. Two pure loads of the same object are concurrent reads.

Happens-before is how the language orders accesses across threads. It is assembled from:

- **sequenced-before** inside one thread (the ordinary left-to-right story of one thread's statements, with the evaluation-order exceptions Week 2 already listed);
- **synchronizes-with**, the edges supplied by thread creation, thread join, mutex unlock/lock pairs, and atomic release/acquire operations.

If A happens-before B, the abstract machine requires B's view of the object to reflect A's side effects. If two conflicting accesses have no happens-before edge either way, the execution has a data race.

```
parent: unsafe_counter = 0
   │
   │  pthread_create          (synchronizes-with thread start)
   ▼
worker 0                          worker 1
load / modify / store  ·······  load / modify / store
        no happens-before edge between these accesses
   │
   │  pthread_join            (synchronizes-with thread termination)
   ▼
parent: printf the counter
```

The vertical edges are real. The parent's store of 0 happens-before every worker's first access, and each worker's termination happens-before the parent's `printf`. The horizontal edge is the one the program never builds. Join orders the printf after the workers finish. The workers' accesses to `unsafe_counter` stay unordered with respect to each other, and those are the conflicting accesses.

Wall-clock overlap is a different question. A scheduler that runs one worker to completion before the other has ordered that run. The abstract machine's promise is the set of edges the program built, and a lucky schedule is not one of those edges.

---

3. The increment in registers

Week 2 spells a memory increment as a load into a register, an `add`, and a store. At `-O0`, GCC 16.2.1 emits that shape for a file-scope `uint64_t`. The rip-relative operand is the relocation for the counter; this object file was not linked, so the displacement prints as zero:

```
mov    0x0(%rip),%rax
add    $0x1,%rax
mov    %rax,0x0(%rip)
```

In the Week 2 spelling those are `movq`, `addq $1, %rax`, and `movq`. The surrounding `-O0` loop keeps the counter in memory on every iteration and the induction variable in a stack slot:

```
movl   $0x0, -0x4(%rbp)          # i = 0
mov    counter(%rip), %rax       # load
add    $0x1, %rax                # add in the register
mov    %rax, counter(%rip)       # store
addl   $0x1, -0x4(%rbp)          # i++
cmpl   $0x30d3f, -0x4(%rbp)      # i <= 199999
jle    ...                       # back to the load
```

`0x30d3f` is 199999, the last value of `i` for which the body runs. `%rax` is private to the core that executes the three memory-touching instructions. The load copies the line's bytes into that register. The add updates the register. The store writes the register back. Between the load and the store the counter in memory is stale relative to `%rax`, and another core is free to own the line.

Take a starting value of 41 and two cores:

```
Core 0                              Core 1
mov  counter(%rip), %rax   # 41     mov  counter(%rip), %rax   # 41
add  $1, %rax              # 42     add  $1, %rax             # 42
mov  %rax, counter(%rip)   # 42     mov  %rax, counter(%rip)  # 42
```

The source asked for 43. Memory holds 42. Each `add` was a correct register operation on a private copy of 41. The same shape with a general starting value `N` stores `N+1` twice: two source-level increments become one surviving store. Nothing in the three-instruction sequence is a single memory operation. The `add` never touches the line.

x86-64 will perform an aligned 8-byte load or store as one architectural access. The read-modify-write is three accesses unless the instruction stream uses a locked read-modify-write. The `-O0` sequence has no `lock` prefix. Day 5's `atomic_fetch_add` is what asks the compiler to emit one.

The prefix is an x86-64 fact. AArch64 compilers emit an atomic add as `LDADD` or as an exclusive `LDXR`/`STXR` pair, depending on the core and the code generator. RISC-V uses an atomic memory operation such as `amoadd.d`. Those are the instructions a C11 atomic asks for. Plain `counter++` asks for none of them: each ISA implements it with an ordinary load, an add in a register, and an ordinary store, which is the sequence above. This lesson quotes the encoding this host emitted.

---

4. Why the cache can look finished while the language is not

Week 2's cache lesson already has the hardware half. A core that wants to write a line takes exclusive ownership; other copies are invalidated or downgraded. After both stores in the diagram above, some core holds the line and every later coherent load sees `N+1`. The protocol did its job. It delivered a single agreed value for a pair of stores that both carried the same bits.

```
counter lives on one 64-byte line

Core 0 load     obtains a readable copy, fills %rax with N
Core 1 load     obtains a readable copy, fills %rax with N
Core 0 store    takes exclusive ownership, line becomes N+1
Core 1 store    takes exclusive ownership, line becomes N+1 again
```

The line never held `N+2`, because `N+2` existed only as a sum of two private registers that were never combined. Coherence orders ownership of the line. The load, the register add, and the store stay three operations.

This is how a non-atomic increment can appear to work. Run the workers with no overlap and each load sees the previous store: the total matches the arithmetic. Run them overlapped and some stores replace other stores of the same computed value: the total comes out low. Both runs used a coherent memory system. The C rule does not consult that outcome. The accesses conflicted, and no happens-before edge ordered them, so the execution has no required meaning either way. A later compiler, a different schedule, or ThreadSanitizer is entitled to treat the same source differently.

False sharing, from the same Week 2 page, is the complementary layout fact: two different objects on one line bounce the line even though they do not conflict in C. Today's bug is true sharing of one object. The line size explains the traffic. The data-race rule explains the meaning.

---

5. The compiler may assume the race does not exist

The as-if rule lets the compiler transform a program when the required observable behavior is preserved. For a program with a data race, there is no required behavior on that execution, so the compiler may transform the code under the assumption that the race does not occur. In practice that assumption is "this thread is the only one performing conflicting accesses on this object for the duration of the loop."

The project's Makefile sets `CFLAGS ?= -O2`. The built `thread_lab` on this host contains, at `increment_unsafe`:

```
400a00:  addq   $0x30d40, 0x2685(%rip)    # unsafe_counter
400a0b:  xor    %eax, %eax
400a0d:  ret
```

`0x30d40` is 200000, the constant `ITERATIONS`. The 200000 separate read-modify-writes of the source have become one add of 200000 to memory. That is a correct compilation of a single-threaded loop: the counter ends 200000 higher, and no conforming program can observe the intermediate values from another thread. It is a correct compilation of the racy program only because the racy program has no requirements. If another thread's increments had to be respected one by one, adding 200000 in one step would drop them on the floor whenever they landed in the middle of the loop.

The same compiler, same version, asked for three nearby shapes:

```
global uint64_t, -O2
    addq   $0x30d40, counter(%rip)          one unlocked add of 200000

volatile uint64_t, -O2
    loop: addq $1, counter(%rip)            a memory RMW per iteration
          no lock prefix

atomic_fetch_add_explicit(..., memory_order_relaxed), -O2
    loop: lock addq $1, counter(%rip)       an atomic RMW per iteration
```

`volatile` keeps the accesses in the instruction stream. Each `addq $1` to memory is still a non-atomic read-modify-write: the prefix that would make it atomic on x86-64 is `lock`, and it is absent. Two cores can still interleave inside that instruction. The abstract machine still sees conflicting non-atomic accesses. `volatile` is the qualifier for objects whose accesses are observable as accesses — device registers, and the narrow signal patterns the standard actually defines. It is a different contract from inter-thread happens-before.

The atomic loop keeps one atomic read-modify-write per source increment (`lock addq` on this ISA; GCC unrolled it by two). `memory_order_relaxed` is enough for the narrow claim that increments are preserved in one modification order. Each operation is one step of that order. Other payload data needs a release/acquire protocol, which day 5 separates from this counter. The final load in the lab happens after every worker has been joined, so thread termination supplies the phase edge around the relaxed adds.

The `-O2` collapse is one compiler's choice. Another version can keep the three-instruction loop. Both are licensed by the same rule. Reading the binary tells you which choice this build made. That choice stays a fact about this build. The C source's meaning is the rule that licensed it.

Even the collapsed `addq $0x30d40, mem` has no `lock` prefix. Four threads each adding 200000 with an unlocked memory read-modify-write can still lose an entire chunk of 200000 when two of those instructions interleave. A racy total below 800000 on this binary is that machine-level loss, or a further consequence of undefined behavior. A racy total of exactly 800000 is a schedule in which those unlocked adds happened not to collide. The source line is the same line in both cases.

---

6. The interleaving picture is already the smaller claim

Section 3 is what the `-O0` instructions do when two cores overlap: each core adds 1 to a private register, and one of the stores survives. It is tempting to stop there, call the bug a lost update, and treat a total of 800000 as proof the schedule was kind. That picture is real, and it is smaller than the permission the compiler used.

The `-O2` function never performs 200000 loads and 200000 stores. The steps an interleaving story wants to shuffle have already been deleted, before any second thread is scheduled. A model that only shuffles source-level `++` operations cannot derive `addq $0x30d40`. Deriving it requires the premise "no other thread conflicts," which is the premise a data race withholds.

The layers sit on top of each other, and each one has an artifact:

```
language    conflicting non-atomic accesses, no happens-before
            → this execution has no requirements

compiler    may delete, combine, or reorder under that permission
            → this binary's worker is one unlocked add of 200000

machine     that addq is still a non-atomic read-modify-write
            → two cores can each read M and each write M+200000
            → the line ends on one of those stores

coherence   the line has one value afterward
            → later loads agree
            → agreement is a hardware fact, and the C execution
              still has no requirements
```

The totals this binary usually prints come from the machine layer: overlapping unlocked adds of 200000 can drop a chunk of that size. Undefined behavior is wider than that menu. The collapse of the loop sits outside the source-level interleaving model. A sanitizer report, a surprising total, or a later compiler that deletes still more of the function are inside the same permission.

The repair restores a requirement, and it is specific. `increment_mutex` performs each update of `mutex_counter` while `counter_lock` is held, so the critical sections are ordered by unlock-then-lock. The relaxed atomic makes each increment one step of a single modification order, and the joins order the final load after those steps. Day 3 and day 5 are those repairs. A kind schedule leaves the missing edge missing. A coherent line leaves it missing too.

---

7. The kernel's own name for an intentional race

The Linux tree you are reading is itself C, and it has the same rule. `include/linux/compiler.h` spells a narrow escape hatch for diagnostics. The comment is the kernel's, and it matches the split above: the macro is a hint to tooling, and it leaves ordinary code generation alone.

```
/* include/linux/compiler.h :: data_race :: tree c9b2385d6793
 * lines 196–214
 */
/**
 * data_race - mark an expression as containing intentional data races
 *
 * This data_race() macro is useful for situations in which data races
 * should be forgiven.  One example is diagnostic code that accesses
 * shared variables but is not a part of the core synchronization design.
 *
 * This macro *does not* affect normal code generation, but is a hint
 * to tooling that data races here are to be ignored.
 */
#define data_race(expr)                                 \
({                                      \
    __unqual_scalar_typeof(({ expr; })) __v = ({            \
        __kcsan_disable_current();              \
        expr;                                   \
    });                                 \
    __kcsan_enable_current();                   \
    __v;                                    \
})
```

`__kcsan_disable_current` tells the kernel's concurrency sanitizer to ignore the expression. The generated instructions for `expr` stay whatever the compiler would have emitted. `copy_process` in the same tree uses the macro on a limit check, `data_race(nr_threads >= max_threads)` at `kernel/fork.c` line 2074: a racy read of a counter, marked on purpose, and not a read-modify-write of user data.

`unsafe_counter++` carries no such mark, and it modifies the object. The abstract-machine rule for an unmarked conflicting pair is the one in section 2. The kernel macro is evidence that the people who compile this tree treat "data race" as a language and tooling category, separate from "the CPU completed a load and a store."

---

8. Execution trace: four workers, one object

`mode_counter("race")` zeroes `unsafe_counter`, starts four threads in `increment_unsafe`, joins them, and prints the value. The binary under discussion is the `-O2` build, so each worker's loop is the single `addq $0x30d40`.

```
parent
  unsafe_counter = 0                          sequenced-before create
  pthread_create × 4                          synchronizes-with each start

worker 0                worker 1              worker 2              worker 3
addq $200000, mem       addq $200000, mem     addq $200000, mem     addq $200000, mem
  |                       |                     |                     |
  └── each addq is an unlocked read-modify-write of the same line
      private ALU work sits between the load half and the store half
      coherence will agree on some final bytes
      no happens-before edge connects any pair of these four accesses

parent
  pthread_join × 4                            synchronizes-with each exit
  printf(race_counter=… expected=800000 data_race=undefined_behavior)
```

One concrete interleaving of two of those `addq` instructions, counter currently 0:

```
Core 0 reads 0, computes 200000
Core 1 reads 0, computes 200000
Core 0 writes 200000
Core 1 writes 200000
line holds 200000; 200000 of source-level increments are gone
```

A different interleaving writes 200000, then 400000, then 600000, then 800000, and the printf shows the expected number. The source, the conflict, and the missing edge are unchanged.

ThreadSanitizer, when the challenge can run it, instruments the accesses and looks for a conflicting pair with no happens-before edge. A report names that pair for the schedule the instrumented binary actually ran. The challenge classifies three other outcomes on purpose: the instrumented program failed to build, the runtime could not execute, or the run finished without a recognized report. Those are facts about the tool on this host. The race is a fact about the source.

---

9. Mastery: Explain · Draw · Observe · Build

Run `../challenges/day-002-data-races.sh`. Predict `mutex_counter=800000`. Leave the racy total unpredicted. Predict that ThreadSanitizer will either report the race or be classified as unavailable or inconclusive. Record which of those the script actually printed.

Explain

- A data race is two conflicting accesses to one non-atomic object, at least one of them a write, with no happens-before edge between them.
- `unsafe_counter++` is a load, an add in a register, and a store. `%rax` on one core is invisible to the other core's `%rax`.
- Coherence makes the cores agree on the line. Agreement on `N+1` after two increments is a coherent machine doing what it was asked. The language assigns that execution no requirements.
- This host's `-O2` build of `increment_unsafe` is `addq $0x30d40` with no `lock` prefix, because the compiler may assume the race does not exist. `volatile` keeps per-iteration accesses and still has no `lock`. `lock addq` is what the relaxed atomic became.
- `pthread_join` orders the printf with thread termination. The missing edge is between the workers.

Draw

1. The happens-before sketch from section 2, with the four workers on the horizontal line and create/join as the only vertical edges.
2. Two cores executing `mov` / `add` / `mov` against one line, and the same two cores executing unlocked `addq $0x30d40`. Mark where the value lives in a register and where it lives in the line.
3. The three instruction shapes: plain `-O2`, `volatile`, and `lock addq`.

Observe

From the challenge, keep:

- the mutex line, which is allowed to be exact;
- the race line, including whatever total this run produced, next to the sentence "this execution has undefined behavior";
- the ThreadSanitizer classification and, if a report was produced, the two stacks it names;
- `cc --version` and the project's `CFLAGS`.

Then disassemble the function the challenge just ran:

```
objdump -d --disassemble=increment_unsafe \
  ../../projects/linux-system-programming/build/thread_lab
```

Record whether you still see one `addq` of `0x30d40` or a load/add/store loop. Rebuild a throwaway copy at `-O0` if you want the three-instruction shape beside it. The addresses and the compiler version belong in the note. A different machine may choose a different shape; the data-race rule is what both shapes are allowed to rely on.

Build

Write a five-line ledger for `unsafe_counter` in the race mode:

```
access              thread        conflicts with          happens-before?
store of 0          parent        later worker accesses   yes: create
load/add/store      worker i      worker j, i != j        no
printf's load       parent        worker stores           yes: join
```

Then answer, in one sentence each: why a printed total of 800000 leaves the ledger's middle row in place, and which instruction in the `-O2` binary is still an unlocked read-modify-write.

Why? notebook

1. Why is "the threads ran at the same wall-clock time" the wrong test for a data race?
2. Why can both cores execute a correct `add` and still lose an increment?
3. Why does a coherent cache line at `N+1` after two increments satisfy the hardware and leave the C program without a meaning?
4. Why is `addq $0x30d40, mem` a legal compilation of a 200000-iteration `++` on a non-atomic object?
5. Why does that same `addq` still have a machine-level race with a second core's `addq`?
6. Why does `volatile` keep the loop and still leave a data race?
7. Why is `pthread_join` enough to order the printf and not enough to order the workers with each other?
8. Why is a ThreadSanitizer skip a fact about the tool, and the race a fact about the source?

References used selectively

- C abstract machine and undefined behavior, as used in Week 2, day 7.
- Linux tree `c9b2385d6793`, `include/linux/compiler.h` `data_race`, and the call at `kernel/fork.c` line 2074.
- Host observation: GCC 16.2.1, `increment_unsafe` in the `-O2` `thread_lab` binary, and the `-O0` / `volatile` / relaxed-atomic shapes compiled beside it.

Next: the mutex mode puts a happens-before edge between those workers. Day 3 names the invariant that edge protects, and the futex that lets the uncontended lock stay in user mode.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Data Races

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-002-data-races.sh
```

- View lab source on GitHub

Day 3 — Mutexes, Ownership, Lock Order, and the Futex

Target time: approximately 3 hours

> `pthread_mutex_lock` returned, and `mutex_counter` went up by one. What invariant was true while the lock was held, who was allowed to touch the object, and why did that lock usually finish without entering the kernel?

Day 2 left four threads conflicting on one non-atomic object, with no happens-before edge between them. The repair in `thread_lab.c` is a mutex around each increment. The lock is there because an invariant needs to stay true across a read-modify-write, and because the unlock has to synchronize with the next locker. The variable is one field inside that invariant. Locking "the counter" is a shorthand for locking the rule the counter is part of.

Kernel excerpts are from tree `c9b2385d6793`, `kernel/futex.c`. The assembly and the futex counts are from one host build of `thread_lab` (GCC 16.2.1, the project's `-O2`) and one `strace` of its `mutex` mode. Counts move with the schedule. The challenge records a fresh trace.

---

1. Why a lock is a statement about an invariant

Write the invariant before choosing the mutex. For this lab it is:

> `mutex_counter` equals the number of increments whose critical sections have completed.

Four threads and 200000 iterations make that number 800000, which is the line the mutex mode is allowed to require. Every read-modify-write of `mutex_counter` in `increment_mutex` happens between `pthread_mutex_lock` and `pthread_mutex_unlock` on `counter_lock`. The critical section in the built binary is a single increment of that object:

```
mov    $0x30d40, %ebx                # 200000
mov    $counter_lock, %edi
call   pthread_mutex_lock
incq   mutex_counter(%rip)
mov    $counter_lock, %edi
call   pthread_mutex_unlock
dec    %ebx
jne    ...                           # back to the lock
```

`incq` is still a read-modify-write. It is safe here because the mutex protocol guarantees that only the holding thread executes it, and that the holder's view of the object includes every increment whose unlock has already synchronized with this lock. The instruction did not become atomic by sitting in a function. The surrounding calls supply the exclusion and the happens-before edge from day 2.

A linked list would have a larger invariant: reachability, link consistency, and the size field, all true at the moment the lock is released. A transfer between two balances would span both balances and the ledger line. In each case the mutex is the name of that whole statement. Taking the lock, updating one field, and dropping the lock before the statement is true again publishes a broken intermediate state to the next locker. Holding the lock across work that is outside the invariant — a blocking read, an unbounded callback — keeps the statement true and stalls every other thread that needs it.

The mutex object itself has a lifetime. `counter_lock` is initialized with `PTHREAD_MUTEX_INITIALIZER` for the life of the process. A mutex that lives inside a heap structure has to be initialized before any thread can lock it, and destroyed only after every lock, unlock, and wait that names it has finished.

---

2. Two kinds of ownership

Mutex ownership. A normal POSIX mutex has one owner while it is locked: the thread whose `pthread_mutex_lock` succeeded. That thread is the thread that calls `pthread_mutex_unlock`. The lab's `increment_mutex` locks and unlocks on the same thread, every iteration. Handing the locked mutex to another thread and unlocking it there is outside the contract of a normal mutex. An error-checking mutex can return a diagnostic for some of those mistakes. The diagnostic is a report. The protocol is still "the locker unlocks."

Object ownership. This answers a different question: which thread may read or write the object, and which thread releases it. The ownership mode in the same file uses no mutex:

```
parent: malloc, item->value = 21
parent: pthread_create(consume_owned, item)     # synchronizes-with start
child:  item->value *= 2, return item           # only the child writes
parent: pthread_join                            # synchronizes-with exit
parent: read 42, free(item)                     # only the parent frees
```

Create and join are the edges from day 1. During the child's execution the parent does not touch `item`. After join the child is gone. The intervals of access do not overlap, so there is no conflicting pair that needs a mutex. The heap allocation stays alive across the whole interval because the parent frees it only after join. Freeing it immediately after `pthread_create` would end the object's lifetime while the child might still be in `consume_owned`. Creation starts the child; it does not promise the child has already read the pointer.

The two words collide on purpose, and they stay separate:

```
mutex owner     the thread inside pthread_mutex_lock / unlock
object owner    the thread the protocol currently allows to touch the bytes
```

The counter's object is shared for the whole run, so its protocol is "the mutex owner may touch it, and only then." The heap item's object owner moves from parent to child at create and back at join, so its protocol needs no lock.

---

3. Where the happens-before edge comes from

Inside one thread, the store to `mutex_counter` is sequenced before the unlock. A mutex unlock **synchronizes with** a later successful lock of that same mutex. Anything sequenced before the unlock happens-before anything sequenced after that later lock.

```
thread A                          thread B
lock(counter_lock)
  incq mutex_counter              .
unlock(counter_lock)  ──────────► lock(counter_lock)
                                  incq sees A's increment
                                  unlock(counter_lock)
```

The arrow is the unlock and lock of `counter_lock`. An unlock of a different mutex leaves `mutex_counter` unpublished for a thread whose next step is to lock `counter_lock`. An uncontended lock still has the edge: the synchronizes-with relation is part of the operation's meaning, whether or not a waiter was asleep.

Day 2's race mode has the create and join edges and lacks this middle arrow. The mutex mode adds the arrow on every iteration. That is why the checked total is a requirement of the program and the racy total is not.

---

4. A global order when one thread needs two locks

One lock has no deadlock cycle. Two locks do, as soon as one path acquires A then B and another acquires B then A. Each holds the lock the other is waiting for.

`deadlock_lab.c` has a path that refuses that cycle. `ordered` always takes `locks[0]` and then `locks[1]`:

```
pthread_mutex_lock(&locks[0]);
pthread_mutex_lock(&locks[1]);
pthread_mutex_unlock(&locks[1]);
pthread_mutex_unlock(&locks[0]);
```

Every caller that needs both locks uses that order, including the second thread the `ordered` mode creates. A rank is the same rule written for more than two locks: acquire in increasing rank, release in reverse, and call out to unknown code only when you can name the locks it might take.

```
legal                         the cycle day 6 studies
thread A    thread B          thread A         thread B
lock 0      lock 0            lock 0           lock 1
lock 1      lock 1            lock 1 (waits)   lock 0 (waits)
unlock 1    unlock 1
unlock 0    unlock 0
```

The challenge runs the left-hand shape and expects `lock_order=0_then_1 completed=yes`. Day 6 runs the right-hand shape with a timed lock so the wait ends and the process exits. A timeout reports that the lock stayed unacquired. Work already performed under the lock that

was acquired remains in place. The order has to hold on the error path too: a function that takes lock 0, fails, and then takes lock 1 while another thread does the opposite has rebuilt the cycle on the failure branch.

Document, for each function, whether it assumes a lock is already held, takes the lock itself, or must be called with no locks held. The lab's `ordered` takes both internally and releases both before it returns.

---

5. Why the fast path stays in user mode

Week 8 measured what a syscall costs: a privilege transition, a register packet, kernel entry, the work, and the return. A mutex taken on every increment of a hot counter would pay that cost 200000 times per thread if each lock and unlock entered the kernel. The user/kernel boundary you already timed would dominate the increment.

The pthread mutex on Linux avoids that for the common case. The mutex word lives in user memory, in the shared `mm` from day 1. The uncontended path is a user-mode atomic update of that word — a compare-and-swap from "free" to "held," or the unlock's store back to "free." `pthread_mutex_lock` returns to `incq` without a syscall. The kernel is entered when a thread must sleep because the word is already held, or when an unlock must wake a thread that went to sleep. That sleep/wake operation is a **futex**.

The split, as a design:

```
lock, word is free
  atomic compare-and-swap in user mode
  return to the critical section          # no syscall

lock, word is held
  mark "waiters" in the word
  futex_wait                              # syscall, sleep

unlock, no waiters
  atomic store of "free"
  return                                  # no syscall

unlock, waiters were marked
  store "free"
  futex_wake                              # syscall, wake one
```

The integers in the word are a libc encoding. In the comment below, the kernel waits and wakes on a user address. The invariant about `mutex_counter` stays in the libc that owns the word.

One run of this host's `thread_lab mutex` printed `mutex_counter=800000`. `strace -f -e trace=futex` on that run recorded 315 `FUTEX_WAIT_PRIVATE`, 555 `FUTEX_WAKE_PRIVATE`, and 2 `FUTEX_WAIT_BITSET` — 872 futex entries against 800000 lock calls and 800000 unlock calls. The waits that named the mutex used the address `increment_mutex` passes to `pthread_mutex_lock` (`0x4030a8` in that binary) and an expected value of 2, which is glibc's "held, and someone may be waiting" encoding on this lib, not a kernel constant. One of those waits returned `EAGAIN`: the kernel re-read the word and found it had already changed, which is

the check the comment in the next section exists to require. Another run will print different counts. The stable fact is the ratio: hundreds of sleeps beside hundreds of thousands of critical sections. The sections that found the word free never appeared in the trace.

---

6. What the kernel promises about wait and wake

`kernel/futex.c` opens with the contract. The waiter samples the user word, then asks the kernel to sleep only if the word is still that value. The waker updates the word, then asks the kernel to wake. Both sides compute a hash bucket and take its lock.

```
/* kernel/futex.c :: file comment :: tree c9b2385d6793
 * lines 47–64
 */
/*
 * READ this before attempting to hack on futexes!
 *
 * Basic futex operation and ordering guarantees
 * =============================================
 *
 * The waiter reads the futex value in user space and calls
 * futex_wait(). This function computes the hash bucket and acquires
 * the hash bucket lock. After that it reads the futex user space value
 * again and verifies that the data has not changed. If it has not changed
 * it enqueues itself into the hash bucket, releases the hash bucket lock
 * and schedules.
 *
 * The waker side modifies the user space value of the futex and calls
 * futex_wake(). This function computes the hash bucket and acquires the
 * hash bucket lock. Then it looks for waiters on that futex in the hash
 * bucket and wakes them.
 *
```

The comment then shows the bug those steps exist to close. A waiter can read the old value, a waker can update the word and observe an empty queue, and the waiter can then enqueue and sleep with nobody left to wake it. The repair is a pair of barriers and a waiter count, so that either the waiter sees the new value and refuses to sleep, or the waker sees the waiter and wakes it:

```
/* kernel/futex.c :: file comment :: tree c9b2385d6793
 * lines 91–121
 */
 * The correct serialization ensures that a waiter either observes
 * the changed user space value before blocking or is woken by a
 * concurrent waker:
 *
 * CPU 0                                  CPU 1
 * val = *futex;
 * sys_futex(WAIT, futex, val);
 *   futex_wait(futex, val);
 *
 *   waiters++; (a)
 *   smp_mb(); (A) <-- paired with -.
 *                                  |
 *   lock(hash_bucket(futex));      |
 *                                  |
 *   uval = *futex;                 |
 *                                  |        *futex = newval;
 *                                  |        sys_futex(WAKE, futex);
 *                                  |          futex_wake(futex);
 *                                  |
 *                                  `--------> smp_mb(); (B)
 *   if (uval == val)
 *     queue();
 *     unlock(hash_bucket(futex));
 *     schedule();                         if (waiters)
 *                                           lock(hash_bucket(futex));
 *   else                                    wake_waiters(futex);
 *     waiters--; (b)                        unlock(hash_bucket(futex));
 *
 * Where (A) orders the waiters increment and the futex value read through
 * atomic operations (see hb_waiters_inc) and where (B) orders the write
 * to futex and the waiters read (see hb_waiters_pending()).
```

Barrier (B) is the `smp_mb` in `hb_waiters_pending` (lines 341–348), which the wake path calls before it decides the queue is empty. Barrier (A) is the matching count increment on the wait path. Together they are the kernel's statement of the lost-wakeup problem you will meet again with condition variables on day 4: the sleep decision and the wake decision have to share an order, or a wakeup is consumed by nobody.

`EAGAIN` from the run in section 5 is this re-read succeeding in the useful direction. The waiter reached the kernel, the word had already moved, and `futex_wait` returned instead of sleeping. The userspace lock path tries the atomic update again.

---

7. One hash bucket for many user addresses

Wait queues live in a fixed table of buckets. `hash_futex` maps a key derived from the user address into that table:

```
/* kernel/futex.c :: hash_futex :: tree c9b2385d6793
 * lines 354–367
 */
/**
 * hash_futex - Return the hash bucket in the global hash
 * @key:    Pointer to the futex key for which the hash is calculated
 *
 * We hash on the keys returned from get_futex_key (see below) and return the
 * corresponding hash bucket in the global hash.
 */
static struct futex_hash_bucket *hash_futex(union futex_key *key)
{
    u32 hash = jhash2((u32 *)key, offsetof(typeof(*key), both.offset) / 4,
              key->both.offset);

    return &futex_queues[hash & (futex_hashsize - 1)];
}
```

Waiters for different user addresses can share a bucket. `match_futex` compares the full key so a wake only unblocks waiters for the address it was asked about. The bucket lock is kernel serialization for the queue. It is a different object from `counter_lock`, which is the user word the critical section is built on.

`futex_wait` is the sleep side. It rejects an empty bitset, prepares an optional timeout, and calls `futex_wait_setup` to re-read the user word and enqueue under the bucket lock. Only then does it schedule:

```
/* kernel/futex.c :: futex_wait :: tree c9b2385d6793
 * lines 2684–2709
 */
static int futex_wait(u32 __user *uaddr, unsigned int flags, u32 val,
              ktime_t *abs_time, u32 bitset)
{
    struct hrtimer_sleeper timeout, *to;
    struct restart_block *restart;
    struct futex_hash_bucket *hb;
    struct futex_q q = futex_q_init;
    int ret;

    if (!bitset)
        return -EINVAL;
    q.bitset = bitset;

    to = futex_setup_timer(abs_time, &timeout, flags,
                   current->timer_slack_ns);
retry:
    /*
     * Prepare to wait on uaddr. On success, holds hb lock and increments
     * q.key refs.
     */
    ret = futex_wait_setup(uaddr, val, flags, &q, &hb);
    if (ret)
        goto out;

    /* queue_me and wait for wakeup, timeout, or a signal. */
    futex_wait_queue_me(hb, &q, to);
```

`uaddr` is the user virtual address of the word. `val` is the value the caller sampled. `flags` carries whether the futex is process-private. `FUTEX_WAIT_PRIVATE` in the trace is that private form: both threads share an `mm`, so the key does not have to be a shared-mapping key.

`futex_wake` is the other side. It builds the key, hashes it, and skips the bucket lock when the waiter count says the bucket is empty — the fast path of the wake, still a syscall, but a short one:

```
/* kernel/futex.c :: futex_wake :: tree c9b2385d6793
 * lines 1586–1611
 */
/*
 * Wake up waiters matching bitset queued on this futex (uaddr).
 */
static int
futex_wake(u32 __user *uaddr, unsigned int flags, int nr_wake, u32 bitset)
{
    struct futex_hash_bucket *hb;
    struct futex_q *this, *next;
    union futex_key key = FUTEX_KEY_INIT;
    int ret;
    DEFINE_WAKE_Q(wake_q);

    if (!bitset)
        return -EINVAL;

    ret = get_futex_key(uaddr, flags & FLAGS_SHARED, &key, FUTEX_READ);
    if (unlikely(ret != 0))
        return ret;

    hb = hash_futex(&key);

    /* Make sure we really have tasks to wakeup */
    if (!hb_waiters_pending(hb))
        return ret;

    spin_lock(&hb->lock);
```

The loop that follows walks `hb->chain` with `match_futex`, calls `mark_wake_futex` for up to `nr_wake` matches, drops the bucket lock, and `wake_up_q`s them. A pthread unlock that knows a waiter exists calls this with `nr_wake` of 1. The woken thread returns to user mode and tries the atomic lock of the word again. Between the wake and that successful compare-and-swap, another thread may have taken the word; the woken thread then waits again. The futex wake is permission to retry, paired with the user-word protocol libc maintains.

---

8. Execution trace: one critical section, then a sleep

One iteration of `increment_mutex` on thread A that finds the word free, overlapping a thread B that finds it held. The word values here are the design in section 5; the trace's expected value of 2 is this libc's "waiters" state, and a different libc can use a different encoding.

```
thread A, user                          thread B, user                  kernel
word == free
CAS free → held
  pthread_mutex_lock returns
incq mutex_counter
                                        CAS fails (word is held)
                                        store "waiters"
                                        syscall futex_wait(word, val)
                                                                        hash_futex(key)
                                                                        re-read *word
                                                                        val still matches
                                                                        enqueue on bucket
                                                                        schedule B out
store free
word says waiters → syscall futex_wake(word, 1)
                                                                        hash_futex
                                                                        hb_waiters_pending
                                                                        bucket lock, match_futex
                                                                        wake B
                                        futex_wait returns
                                        CAS free → held
                                        incq mutex_counter
                                        unlock
```

A's `incq` and B's `incq` are in different critical sections. A's unlock synchronizes with B's later successful lock, so B's `incq` happens after A's. The `incq` instructions themselves are ordinary. The edge is the mutex.

The iterations where B's CAS succeeds never appear as a `futex` line. That absence is the point of the design: the syscall exists for the sleep, and the uncontended path never enters the kernel. A trace full of `FUTEX_WAKE_PRIVATE` is evidence about the contended tail of that run. The line count beside 800000 lock calls describes the slow path. One logical wait can show up as an unfinished line and a resumed line under `strace -f`, and an unlock with no sleeper never calls `futex` at all. The challenge's limit line says the same thing. Use the trace to see the boundary. Use the source to see the invariant.

---

9. Mastery: Explain · Draw · Observe · Build

Run `../challenges/day-003-mutex-ownership.sh`. Predict `mutex_counter=800000`, `joined_value=42` with `freed_by=parent`, and `lock_order=0_then_1 completed=yes`. If `strace` exists, the script keeps a futex trace. Predict that the trace is much smaller than 1.6 million lines, and leave the exact count unpredicted.

Explain

- The lock protects an invariant. For the lab, `mutex_counter` equals the number of completed critical sections, and the `incq` is one step inside that statement.
- Mutex ownership is "the thread that locked unlocks." Object ownership is "who may touch the bytes, and when." The heap transfer uses create and join because those intervals do not overlap.
- Unlock of a mutex synchronizes with a later lock of that same mutex.
- `ordered` takes `locks[0]` before `locks[1]` on every path. That order is what day 6's opposite-order cycle lacks.

- The uncontended lock is a user-mode atomic on a word in the shared `mm`. `futex_wait` and `futex_wake` are the sleep and the wakeup. `hash_futex` places both on a bucket in `futex_queues`. The kernel re-reads the user word under the bucket lock so a wake and a wait cannot miss each other.

Draw

1. The invariant as a sentence, and every source line that reads or writes `mutex_counter`, each marked "lock held" or "lock not required because this thread is the only one alive in this interval."
2. The object-ownership timeline for `item` from `malloc` to `free`.
3. Two threads and two locks in index order, beside the cycle where each holds the lock the other waits for.
4. The uncontended CAS and the contended `futex_wait` / `futex_wake` path, with the user/kernel boundary drawn on the syscall only.

Observe

From the challenge output, keep:

- `mutex_counter=800000 expected=800000`;
- `ownership=heap_transfer joined_value=42 freed_by=parent`;
- `lock_order=0_then_1 completed=yes`;
- if a trace was captured, the path the script prints, the operation names (`FUTEX_WAIT_PRIVATE`, `FUTEX_WAKE_PRIVATE`, anything else), and whether any wait returned `EAGAIN`.

Then:

```
objdump -d --disassemble=increment_mutex \
  ../../projects/linux-system-programming/build/thread_lab
```

Confirm the `incq` of `mutex_counter` sits between the lock call and the unlock call. The futex lines, if any, are the slow path inside those calls. A trace with zero futex lines on a quiet machine is still compatible with 800000 critical sections that all took the fast path.

Build

Write the invariant of `counter_lock` in one sentence, and the ownership rule of `item` in one sentence. Then, from the trace or from the source of `ordered`, list the lock ranks:

```
rank 0    locks[0]     acquired first by every path that needs both
rank 1    locks[1]     acquired second, released first
```

State what would make a third lock legal to take while rank 1 is held (a higher rank, on every path), and what day 6 will demonstrate by taking them in opposite orders.

Why? notebook

1. Why is "I locked around the variable" weaker than "this lock protects this invariant"?
2. Why does the ownership mode free `item` after join and use no mutex?

3. Why does unlocking mutex A fail to publish a write that the other thread observes by locking mutex B?
4. Why can an uncontended lock still be the happens-before edge?
5. Why does a syscall on every lock throw away the user/kernel boundary Week 8 measured?
6. Why does `futex_wait` re-read the user word after it has taken the hash-bucket lock?
7. Why can two unrelated user addresses share a bucket, and how does a wake still find the right waiters?
8. Why is a futex count of a few hundred, next to 800000 locks, evidence about the slow path and silent about each fast-path acquisition?

References used selectively

- Linux tree `c9b2385d6793`, `kernel/futex.c`: the wait/wake comment at lines 47–122, `hash_futex`, `futex_wait`, `futex_wake`, `hb_waiters_pending`.
- Day 2's missing edge and Week 8's syscall boundary.

Next: day 4 waits for a condition with the same lost-wakeup shape the futex comment just drew, and it re-checks the predicate under the mutex after every wake.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Mutex Ownership

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-003-mutex-ownership.sh
```

- View lab source on GitHub

Day 4 — Condition Variables and Bounded Queues

Target time: approximately 3 hours

> A mutex can make a predicate check true of the protected state. It cannot, by itself, put a thread to sleep until that predicate becomes true. A condition variable exists to close that gap.

Run the day's experiment before the kernel excerpts, and keep the two layers apart. The lab speaks POSIX through glibc. The excerpts later in this lesson are Linux 5.10.134 at tree `c9b2385d6793`, and they show how the kernel waits. glibc's `pthread_cond_t` is a userspace object. This lesson quotes kernel code only.

1. Why exclusion is the wrong tool for waiting

Day 3 gave a mutex a job: name an invariant, and hold the mutex across every read and write that must appear atomic with respect to other threads. For the queue in today's lab the invariant is small and strict. `queue.count` is the number of live slots in a ring of eight, `head` names the oldest slot, and `tail` names the next free slot. While `queue.lock` is held, those fields move together. A producer that sees `count == 8` knows the ring is full *as of that critical section*. A consumer that sees `count == 0` knows the ring is empty as of that same section.

The next question is what the thread should do when the predicate it needs is false.

Spinning while still holding the mutex deadlocks the protocol. The producer is the only thread allowed to increment `count`, and it can increment `count` only while it holds `queue.lock`. A consumer that keeps the lock and rereads `count` has reserved the core's registers for a load that cannot change. The cache line that holds `count` stays hot in that core, and the producer cannot perform the store that would make the load return nonzero.

Dropping the mutex and spinning in userspace fixes the exclusion problem and creates two new ones. The waiter burns a hardware thread: its loop occupies an instruction pointer, a stack, and a share of the memory hierarchy, which matters as soon as the machine has more waiters than idle cores. And there is a gap between "I observed false" and "I am asleep." In that gap the producer can fill the queue and issue a notification that nobody is waiting to hear.

Sleeping without a protocol is the same gap, moved into the kernel. A thread that decides to block has to publish "I am a waiter" and release the mutex as one transition. If those are separate steps, a wakeup can land after the decision and before the publication. The waiter then enters the scheduler with a predicate that is already true and a notification that has already been consumed by an empty wait list.

That is why a condition variable is a different object from a mutex. The mutex protects the predicate. The condition variable is the list of threads that have committed to sleep until a later critical section asks them to look again. POSIX packages the commit as `pthread_cond_wait`: the caller holds the mutex; the call releases it and blocks; the call returns only after the mutex has been reacquired.

```
mutex answers:  "Is the protected state consistent while I look?"
condvar answers: "Wake me when someone else may have changed it."
predicate answers: "May I proceed on this look?"
```

The three answers stay separate. A signal is a hint that the predicate might now hold. The bytes of `count` are the predicate. The mutex is what makes a read of those bytes agree with the writer that last updated them.

2. Where the queue lives, in the machine

The lab queue is one object in the process address space. Both producers and both consumers use the same virtual addresses for `items`, `head`, `tail`, `count`, the mutex, and the two condition variables. Sharing the mapping is what Day 1 established: `pthread_create` on Linux does not give the new thread a private heap. Virtual memory still translates each thread's loads and stores through the same page tables, so a store to `count` by the producer is a store to the physical page the consumer will load.

Each thread keeps a private stack. `put` and `get` push a return address and spill caller-saved registers on the calling thread's stack. When a consumer blocks inside `pthread_cond_wait`, that frame remains mapped. The kernel saves the userspace registers at the syscall boundary, switches to another task's register state, and later restores this consumer so `pthread_cond_wait` can return into the `while` that called it. Sleep is a scheduling event. It is not an unmap, and it is not a destruction of the C automatic objects in the waiter.

The C abstract machine sees a narrower fact. Every read and write of `count`, `head`, `tail`, and `items` in the lab happens while `queue.lock` is held, and the unlock that ends one critical section synchronizes with the lock that begins the next. Those accesses are therefore ordered by happens-before. Day 2 is why that sentence matters: the same bytes, read and written with no such edge, would be a data race and the program would have undefined behavior. The condition variable does not relax that rule. It adds a place to sleep between critical sections.

On the uncontended fast path a pthread mutex is a userspace atomic on a word in that shared object. On the slow path glibc asks the kernel to block the task with a futex operation. Either way, the POSIX rule the lab relies on is the same: `pthread_cond_wait` returns with the mutex owned by the caller, and the predicate is examined again under that ownership.

3. The wait protocol

Write the protocol as four steps that share one mutex. The names below match `put` and `get` in the lab.

1. Acquire the mutex that protects the predicate.
2. While the predicate is false, wait on the condition variable, passing that same mutex. The wait releases the mutex and sleeps as one operation, then reacquires the mutex before it returns.
3. With the mutex held and the predicate true, perform the state transition.
4. If the transition may have made some other predicate true, signal the condition variable that names it, then release the mutex.

Step 2 is a loop. The call returns holding the mutex, which means the waiter may evaluate the predicate again in the same critical section that will do the transition. An `if` around the wait evaluates the predicate once. The `while` in the lab evaluates it every time the wait returns.

```
lock(M)
while predicate is false:
    cond_wait(CV, M)          # unlock M, sleep, lock M, return
transition protected state
signal(the other CV)          # still holding M, in this lab
unlock(M)
```

The atomicity that matters is the pair "release M" and "become a published waiter." POSIX assigns that pair to `pthread_cond_wait`. A program that unlocks by hand and only then calls the wait has opened the gap itself.

Signaling while still holding the mutex, as `put` and `get` do, keeps the state transition and the notification inside one critical section. The woken thread cannot observe the new `count` until it acquires the mutex, and by then the transition has already happened. Signaling after the unlock is a different schedule with the same logical outcome when the waiter uses the loop above: either the waiter has not yet called `cond_wait`, in which case it will lock, see the updated predicate, and skip the wait, or it is already inside `cond_wait`, in which case it is on the wait list and the signal finds it. The lab takes the simpler side of that choice and signals before unlock.

A condition variable stores waiters. A semaphore stores permits. `pthread_cond_signal` with an empty wait list returns having woken nobody, and the next thread to wait does not inherit that call. A semaphore can do the opposite: a `post` with no waiter increments a count that a later `wait` consumes. Treating a condvar as a semaphore is how notifications get lost. The queue does not need a stored permit. `count` itself is the memory of how many items are present.

4. Execution trace: signal before anybody waits

This is the lost-wakeup trace. The mutex is free, `count` is 0, and the condition variable's waiter list is empty. Thread S signals without holding the mutex. Thread W waits afterwards. No second signal arrives.

```
time  mutex owner   count   waiter list    what runs
----  ------------  ------  -------------  --------------------------
 1    nobody        0       empty          S calls pthread_cond_signal
 2    nobody        0       empty          signal finds no waiter; returns
 3    W             0       empty          W locks and reads count == 0
 4    (inside wait) 0       W              W enters pthread_cond_wait,
                                           releases the mutex, sleeps
 5    nobody        0       W              S has already returned and
                                           does not signal again
```

Walk the machine, not only the API.

At time 1, S's call enters glibc. The userspace waiter list for this condition variable is empty, so glibc has no task to hand to a futex wake. The call returns to S. Nothing in the condvar retains the fact that a signal occurred. S never wrote `count`. The predicate is still false, and the notification has already vanished.

At time 3, W acquires the mutex. That acquisition is a userspace atomic, and on this schedule it is uncontended, so it may complete without a futex syscall. W's load of `count` happens while W owns the mutex. The load returns 0. The predicate W needs, `count > 0`, is false.

At time 4, W calls `pthread_cond_wait`. This is the moment the protocol would have been safe had the signal still been in the future. W publishes itself on the waiter list and releases the mutex, then enters the kernel to block. The core that was running W saves W's userspace registers and the kernel picks some other task. W's stack still holds the frame that will return into `get`. The virtual page that holds `count` stays mapped.

At time 5 the scheduler has a sleeping task whose wake condition will be a later `pthread_cond_signal` or an equivalent glibc wake. S has none left to send. W stays off the run queue. The queue is empty and a consumer is asleep, which is a consistent state, and it is also a stuck state: the consumer is waiting for a notification that already happened to an empty list.

Two repairs are easy to confuse.

If S, still at time 1, had held the mutex, set `count` to 1, and signaled, the signal would still find an empty list and would still be discarded. W's later lock-and-check would then observe `count == 1` and would skip the wait. The predicate, updated under the mutex, is what W observes. The early signal is redundant once the predicate is true at the check. The bug in the trace is the combination of a signal that matches no waiter and a predicate that is still false when W finally waits.

If W had unlocked after seeing 0, S had then locked, set `count` to 1, signaled, and unlocked, and W had later waited *without looking again*, the same stuck state would appear for a different reason. The signal in the gap matched no waiter, and W slept over a predicate that had become true. The `while` loop closes that variant: after W reacquires the mutex it loads `count`, sees 1, and takes the item. `pthread_cond_wait` closes the narrower gap inside a single call, where the unlock and the publication of the waiter are one operation. The loop closes every other reason the call might return with the predicate still false, and it also closes the case where the predicate became true and was consumed again before this thread ran.

Calling `pthread_cond_signal` without the mutex is legal POSIX when the program still updates the predicate under the mutex and every waiter rechecks in a loop. The trace above is the case that loses: the signal is early, the predicate is not true at the eventual check, and nobody signals again.

5. Spurious wakeups and stolen wakeups

A wait can also return when a signal did happen, and the waiter must still recheck.

POSIX allows `pthread_cond_wait` to return when no thread has signaled or broadcast. That return is a spurious wakeup. The mutex is held again. The predicate may still be false. An implementation can produce this from a kernel wakeup that was not the logical event, from an internal race between the userspace word and the futex queue, or from a path that must leave the sleep for a reason the application did not name. Application code treats the return as "look at the predicate," because that is the only fact the application owns.

A stolen wakeup is a real signal whose resource is gone by the time this thread looks. Consider two consumers and one item, which is the shape of the lab whenever the ring holds a single slot and both consumers are in `get`.

```
count == 0. C1 and C2 are inside cond_wait on not_empty.
P locks, writes one item, count = 1, signals not_empty, unlocks.
The signal wakes C1.
C2 is also runnable: it was between loop iterations, or an earlier
broadcast woke it, or it has just arrived at the lock.
C2 acquires queue.lock first, observes count == 1, takes the item,
sets count = 0, signals not_full, unlocks.
C1 acquires queue.lock, observes count == 0, and waits again.
```

C1 was woken for an item that C2 now owns. The signal was honest at the time P issued it. Honesty at signal time is not a reservation. `pthread_cond_signal` wakes at least one waiter; it does not transfer ownership of a queue slot, and it does not hand the mutex to a chosen thread as a Hoare-style monitor would. The lab's semantics are Mesa-style: the signaler continues, the waiter reacquires, and the predicate is re-evaluated under the lock. The `while` is the entire handling of both the spurious return and the stolen item.

`pthread_cond_broadcast` wakes every waiter, which makes the stolen-wakeup schedule ordinary rather than exotic. The lab uses `pthread_cond_signal` because each successful `put` creates one item and each successful `get` creates one free slot. One new right-to-proceed is enough to wake one thread, and the loop covers the case where that thread is not the one that consumes the right. Broadcast would also be correct here. It would schedule more threads back onto the mutex, most of which would observe a false predicate and sleep again.

6. Empty and full are the two predicates

The lab ring has `CAPACITY` 8. Two producers each insert 1000 positive values. Two consumers remove values until they see a sentinel. The fields under `queue.lock` are:

```
items[8]   the slots
head       index of the oldest item
tail       index of the next store
count      number of occupied slots, 0 .. 8

can_get    count > 0      waited on not_empty
can_put    count < 8      waited on not_full
```

`put` locks, waits while `count == 8`, writes `items[tail]`, advances `tail` modulo 8, increments `count`, signals `not_empty`, and unlocks. `get` locks, waits while `count == 0`, reads `items[head]`, advances `head`, decrements `count`, signals `not_full`, and unlocks. Both transitions preserve `0 <= count <= 8` and keep `head` and `tail` inside the ring. The occupied range is the `count` slots beginning at `head`, wrapping at 8. That sentence is the invariant the mutex protects.

A picture of a ring that holds three items, with two producers asleep because the picture is about to fill, helps separate the pointers from the count:

```
index:   0     1     2     3     4     5     6     7
       [ 41 ][ 42 ][ 43 ][    ][    ][    ][    ][    ]
         ^head             ^tail
         count == 3

after five more successful puts, count == 8, tail == head:
a put must wait on not_full
a get waits on not_empty only when count == 0
```

When `count` is 0, `head == tail` and the ring is empty. When `count` is 8, `head == tail` and the ring is full. The pointers alone do not say which. `count` is part of the predicate, which is why every wait expression in the lab reads `count` and not a comparison of `head` and `tail`.

Producers 0 and 1 pass `long` ids 0 and 1. Producer `id` stores `id * 1000 + i` for `i` from 1 to 1000, so the values are 1..1000 and 1001..2000. The parent checks that the consumers together removed `n = 2000` values whose sum is `n * (n + 1) / 2 = 2001000`, and that `count` is 0 at the end. The ring is FIFO for the queue as a whole: each `put` appends and each `get` removes the oldest slot, so one producer's values stay in order relative to each other, interleaved with the other producer's values according to who won the mutex. The checked sum detects many dropped, duplicated, and corrupted payloads. It does not record that interleaving, and the wrapper says the wake order is unspecified.

Capacity 8 is small on purpose. Producers must sometimes observe `can_put` false and enter `pthread_cond_wait` on `not_full`. Consumers must sometimes observe an empty ring, including at startup if they run before any `put`. A huge buffer could let all 2000 inserts finish before a consumer ran, and the wait path would be unexercised even though the final sum would match.

The signals match the predicates they publish. A `put` that increments `count` from 0 to 1 makes `can_get` true, so it signals `not_empty`. A `get` that decrements `count` from 8 to 7 makes `can_put` true, so it signals `not_full`. Signaling the other way would wake a thread whose predicate is still false; the loop would send it back to sleep, and the thread that could make progress would stay on the other wait list until some later signal.

7. Shutdown and who owns the result

After both producers have been joined, the parent calls `put(-1)` twice, once per consumer. The joins matter. They are a happens-before edge from each producer's last `put` to the parent's sentinels, so the sentinels enter the ring after every positive value that producer was going to write. Consumers treat `-1` as "stop," and they do not add it into the sum or the count.

Each consumer `calloc`s a `struct result`, fills `sum` and `count`, and returns the pointer from its thread function. The parent `pthread_join`s that pointer, folds it into the totals, and `free`s it. Ownership is the same pattern as Day 3: the worker allocates, the join publishes the pointer to the parent, and the parent is the only thread that frees it. The queue mutex is not involved in that free. By the time the parent reads the result, the consumer has returned and the join has synchronized with that return.

The sentinel path can block. If the ring is full of positive values, `put(-1)` waits on `not_full` until a consumer takes something. That is ordinary `can_put`. It is also why the sentinels are inserted only after the producers have finished: a sentinel occupying a slot cannot be overtaken by a later positive `put` from a producer that is still running, because those producers have already been joined.

8. What this kernel tree does when a task waits

glibc implements `pthread_cond_wait`, `pthread_cond_signal`, and the mutex operations. On Linux the blocking path is a `futex` system call. The call crosses the privilege boundary: userspace `RIP` enters the kernel through the architecture's syscall instruction, the kernel sees

a task that wants to sleep on a userspace word, and `schedule` runs something else on that core. The excerpts below are that kernel mechanism at tree `c9b2385d6793`. They are not a line-by-line listing of glibc, and this lesson does not pretend to have read the nptl sources.

The kernel's own "sleep until a C expression is true" macro is `wait_event` in `include/linux/wait.h`. It checks the expression, and if the expression is false it goes to `__wait_event`, whose body (the `___wait_event` macro just above this excerpt) is a loop: prepare to wait, test the condition, and only then call `schedule()`.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: include/linux/wait.h
 * lines: 318–336
 * function: wait_event
 */
/**
 * wait_event - sleep until a condition gets true
 * @wq_head: the waitqueue to wait on
 * @condition: a C expression for the event to wait for
 *
 * The process is put to sleep (TASK_UNINTERRUPTIBLE) until the
 * @condition evaluates to true. The @condition is checked each time
 * the waitqueue @wq_head is woken up.
 *
 * wake_up() has to be called after changing any variable that could
 * change the result of the wait condition.
 */
#define wait_event(wq_head, condition)                      \
do {                                            \
    might_sleep();                                  \
    if (condition)                                  \
        break;                                  \
    __wait_event(wq_head, condition);                   \
} while (0)
```

The comment states the same split as the lab. Something must change the variables that the condition reads, and `wake_up` must be called after that change. The macro rechecks. `might_sleep()` is a kernel annotation that this path can block; it is not a userspace condvar.

The order between "I am about to sleep" and "I looked at the condition" is the subject of the comment on `set_current_state` in `include/linux/sched.h`. `prepare_to_wait_event` in `kernel/sched/wait.c` calls `set_current_state` while holding the wait-queue lock, before the waiter returns to the loop and evaluates the condition. The comment explains why that state store carries a memory barrier, and why a colliding wakeup store is repaired by going around the loop again.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: include/linux/sched.h
 * lines: 160–193
 * function: set_current_state
 * The comment continues briefly after this excerpt and closes at line 196.
 */
/*
 * set_current_state() includes a barrier so that the write of current->state
 * is correctly serialised wrt the caller's subsequent test of whether to
 * actually sleep:
 *
 *   for (;;) {
 *  set_current_state(TASK_UNINTERRUPTIBLE);
 *  if (CONDITION)
 *     break;
 *
 *  schedule();
 *   }
 *   __set_current_state(TASK_RUNNING);
 *
 * If the caller does not need such serialisation (because, for instance, the
 * CONDITION test and condition change and wakeup are under the same lock) then
 * use __set_current_state().
 *
 * The above is typically ordered against the wakeup, which does:
 *
 *   CONDITION = 1;
 *   wake_up_state(p, TASK_UNINTERRUPTIBLE);
 *
 * where wake_up_state()/try_to_wake_up() executes a full memory barrier before
 * accessing p->state.
 *
 * Wakeup will do: if (@state & p->state) p->state = TASK_RUNNING, that is,
 * once it observes the TASK_UNINTERRUPTIBLE store the waking CPU can issue a
 * TASK_RUNNING store which can collide with __set_current_state(TASK_RUNNING).
 *
 * However, with slightly different timing the wakeup TASK_RUNNING store can
 * also collide with the TASK_UNINTERRUPTIBLE store. Losing that store is not
 * a problem either because that will result in one extra go around the loop
 * and our @cond test will save the day.
```

Read the parenthetical in the middle with the lab in mind. When the predicate test, the predicate update, and the wakeup are all done under one lock, the lock supplies the order and the kernel can use the barrier-free store. A pthread mutex is that lock for `count`. The futex path still has to publish the waiter and recheck the userspace word, because the kernel does not hold `queue.lock` across the sleep. The comment's last sentences are the kernel's version of "the loop saves you": a wakeup store and a sleep store can pass each other, the task may run an extra iteration, and the condition expression is what decides whether to sleep again. Day 5 names the barriers this comment is pointing at.

The futex syscall documents the same race at the userspace boundary. `futex_wait_setup` in `kernel/futex.c` locks the hash bucket, then reads the userspace word, and only then is the waiter eligible to be queued.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: kernel/futex.c
 * lines: 2635–2652
 * function: futex_wait_setup
 */
    /*
     * Access the page AFTER the hash-bucket is locked.
     * Order is important:
     *
     *   Userspace waiter: val = var; if (cond(val)) futex_wait(&var, val);
     *   Userspace waker:  if (cond(var)) { var = new; futex_wake(&var); }
     *
     * The basic logical guarantee of a futex is that it blocks ONLY
     * if cond(var) is known to be true at the time of blocking, for
     * any cond.  If we locked the hash-bucket after testing *uaddr, that
     * would open a race condition where we could block indefinitely with
     * cond(var) false, which would violate the guarantee.
     *
     * On the other hand, we insert q and release the hash-bucket only
     * after testing *uaddr.  This guarantees that futex_wait() will NOT
     * absorb a wakeup if *uaddr does not match the desired values
     * while the syscall executes.
     */
```

That comment is the lost-wakeup trace translated into a hash-bucket lock. If the kernel tested `*uaddr` and only afterwards locked the bucket, a waker could change the word and issue `futex_wake` in between. The wake would find no queued waiter. The waiter would then queue and block on a word that no longer matches. Locking first, testing second, and queueing only if the value still matches is the kernel closing the gap that `pthread_cond_wait` closes for the mutex. The userspace sketches in the comment are the contract a library uses. They are still kernel commentary, not glibc source.

After `futex_wait_queue_me` returns, `futex_wait` may observe a wakeup that was not the event the caller asked for. The function retries.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: kernel/futex.c
 * lines: 2708–2725
 * function: futex_wait
 */
    /* queue_me and wait for wakeup, timeout, or a signal. */
    futex_wait_queue_me(hb, &q, to);

    /* If we were woken (and unqueued), we succeeded, whatever. */
    ret = 0;
    /* unqueue_me() drops q.key ref */
    if (!unqueue_me(&q))
        goto out;
    ret = -ETIMEDOUT;
    if (to && !to->task)
        goto out;

    /*
     * We expect signal_pending(current), but we might be the
     * victim of a spurious wakeup as well.
     */
    if (!signal_pending(current))
        goto retry;
```

The comment names a spurious wakeup in the kernel's own wait path. The retry goes back through `futex_wait_setup`, which reads the userspace word again. That is the same shape as the lab's `while`: leave the sleep, look at the predicate under the protocol's lock, and sleep again if it is still false. glibc has to do an analogous recheck of the pthread predicate. The lines above are the kernel doing it for a futex word.

One more boundary is worth keeping visible. An uncontended `put` on a non-full queue may perform the mutex lock and unlock entirely in userspace and never enter `futex`. A trace that shows no `futex` lines has shown that those particular calls took the fast path. A trace that shows `futex` lines has shown that those calls blocked or woke. Day 3 already treats syscall traces as evidence of the slow path, with a scope limited to the calls that happened.

9. Mastery: Explain · Draw · Observe · Build

The exercise is challenges/day-004-condition-queue.sh. It builds the canonical lab and runs `queue_lab`, then prints the `put`/`get` transitions and the shutdown path. Predict 2000 items, sum 2001000, a final `count` of 0, and the marker `condition_waits=while_predicate_loops`. Leave wake order, the number of waits, and whether any return was spurious as unspecified.

Explain

- Why a consumer that holds `queue.lock` and spins on `count == 0` prevents the store that would make the predicate true.
- What `pthread_cond_wait` does with the mutex on entry and on return, and why the publication of the waiter has to be in the same step as the unlock.
- Why the lost-wakeup trace in section 4 stays asleep, and which fact would have to change (a later signal, or a predicate that is already true at the check) for the consumer to proceed.
- Why a stolen wakeup still has a correct signal in its past, and why the `while` is what makes that schedule safe.
- Which predicate `not_empty` serves, which predicate `not_full` serves, and why `head == tail` is ambiguous unless `count` is known.

Draw

Draw the ring of eight with `head`, `tail`, and `count` for empty, full, and three occupied slots. Under it, draw two wait lists, `not_full` and `not_empty`, and mark which predicate each list is sleeping on. Then draw the five-step lost-wakeup trace with columns for mutex owner, `count`, and waiter list.

Observe

Run the wrapper from the week directory. Record the checked line, the compiler and kernel identity the rest of the week is using, and the source the wrapper prints for the wait, the transition, the signal, the sentinels, the joins, and the frees. In your notes, point at the `while` and write the predicate it reloads.

Build

Using the printed lab source as the specimen, write a one-page account of a single item's life: which thread stores it, which mutex edge publishes that store, which signal is a hint, which thread's `while` observes `count > 0`, and which `free` runs only after `pthread_join`. Include

the sentence the wrapper prints as its limit: wake order, forced spurious wakeups, cancellation, producer-failure recovery, and a schedule proof are outside what this run establishes.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Condition Queue

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-004-condition-queue.sh
```

- View lab source on GitHub

Day 5 — C11 Atomics and Memory Ordering

Target time: approximately 3 hours

> Atomicity says a single object moves from one value to the next with no torn intermediate. Ordering says which *other* accesses in this thread become visible to a thread that observes that value. The C memory orders exist because those are different promises, and because the compiler and the CPU can each rearrange work the source text wrote in one sequence.

The lab's relaxed counter and the lab's relaxed litmus use the same header, `<stdatomic.h>`, and they establish different facts. challenges/day-005-atomics-memory-order.sh checks that the counter equals 800000, runs the litmus, and writes one compiler's assembly for this host. Kernel barrier headers from tree `c9b2385d6793` (Linux 5.10.134) show how *that kernel* lowers its own release, acquire, and full barriers on x86, AArch64, and RISC-V. They are evidence about the kernel. They are not a dump of what `memory_order_release` compiles to in the lab.

1. Why the queue needed a mutex and the counter might not

Day 4 protected four fields that move together: a slot, `head` or `tail`, `count`, and the decision to signal. No single atomic store can publish "the slot is written and `count` has been incremented" as one modification of one object. The mutex made the whole transition one critical section, and its unlock synchronized with the next lock.

A counter that only ever goes up by one is a different shape. Each step reads one integer and writes that integer plus one. If those read-modify-write steps are atomic operations on one atomic object, the C memory model gives them a single modification order. The lab's `atomic_fetch_add_explicit(..., memory_order_relaxed)` is that step. Four threads perform it 200000 times each. The parent joins them and then loads the counter. The checked result is 800000.

The mutex version of the same counter is still correct, and Day 3 uses it to show a critical section. The atomic version is correct for a narrower contract: one object, one modification order, no second piece of payload published by the increment. Choosing relaxed is part of stating that narrower contract. Relaxed does not say "this increment is unordered with respect to itself." It says "do not create a synchronizes-with edge to any other memory."

Day 2 is the reason an ordinary `uint64_t` increment is not a substitute. Conflicting non-atomic accesses with no happens-before edge are a data race, and a data race is undefined behavior. Declaring the object `_Atomic`, or accessing it through the `atomic_*` functions, makes the conflicting accesses atomic. The program stays inside the memory model. What the model then guarantees depends on the memory order written at each access.

2. Modification order

For each atomic object the model supplies one total order of all the modifications of that object. Every thread that looks at the object agrees on that order. It is called the modification order. It is per object. The modification order of `x` and the modification order of `y` are two orders. Nothing in the relaxed fragment of the model pastes them into one timeline.

A read-modify-write such as `atomic_fetch_add` reads the value immediately preceding its own write in that order and computes the new value from it. If the counter starts at 0 and eight hundred thousand relaxed fetch-adds each add 1, the modification order is a chain:

```
0 → 1 → 2 → … → 800000
    ^    ^
    each arrow is one atomic_fetch_add in the modification order
    the thread that performs it is whichever thread the implementation
    serialized at that step
```

The numeric result does not record which thread won which step. It records that every step saw the previous value and wrote the next one, so no increment was lost and no increment was applied twice. That is atomicity of one object. It is the whole of what `memory_order_relaxed` contributes to this counter.

The parent observes 800000 because of a second fact sitting next to the modification order. Each worker's fetch-adds are sequenced before the worker returns. Thread termination synchronizes with `pthread_join`. The parent's load is sequenced after the joins. Happens-before therefore runs from the increments to the load, and the load reads the value those increments produced. A relaxed load in a thread that had no such edge could observe an earlier value in the same modification order. The lab does not do that. It joins, then loads.

Coherence of a single object is easy to picture and easy to over-read. Cores and caches also keep a coherent picture of a cache line, which Day 6 connects to Week 2. Coherence of a line is a hardware property. Modification order is a rule of the C abstract machine. A program can have a coherent machine and still have undefined behavior if it races on a non-atomic object. A program can have a well-defined relaxed counter and still allow surprising interleavings of *other* objects. The litmus in section 7 is that second situation.

3. What `memory_order_relaxed` promises

`memory_order_relaxed` is the order that adds atomicity and participation in the object's modification order, and stops there. A relaxed store and a relaxed load of the same object will agree with that order: a load sees some value that belongs in it, and the usual coherence rules stop a single thread from observing the order backwards. There is no synchronizes-with edge between the store and the load merely because both were relaxed.

Consequences that the lab relies on:

- The fetch-add counter needs no extra ordering because it publishes nothing beside the counter.
- The compiler may reorder a relaxed atomic with neighboring ordinary accesses when the memory model allows it. Sequenced-before inside one thread still governs that thread's own uses of the values it computed. It does not, by itself, publish those ordinary accesses to another thread.

- The CPU may reorder the instructions the compiler emitted, within the architecture's memory model. Relaxed is a request that the compiler emit the cheapest mapping that still implements an atomic access.

`memory_order_relaxed` is the wrong order when a store is the publication of some other bytes. The flag example in section 5 is the standard shape. A relaxed store to `flag` after a plain store to `data` does not make a relaxed load of `flag` synchronize with the writer. A reader that sees `flag == 1` and then reads `data` has no happens-before edge to the writer's store of `data`. If `data` is non-atomic, that is a data race. If both are atomic and relaxed, the program is defined and the load of `data` may still return the old value.

4. The compiler reorders, and then the CPU reorders

Two different machines sit between the source text and the wires.

```
C abstract machine
  sequenced-before inside one thread
  synchronizes-with from atomics, mutexes, thread create/join
  happens-before, modification order, data-race freedom
        |
        |  the compiler must preserve this, and may rearrange
        |  anything the model leaves free
        v
assembly for one ISA
  registers, loads, stores, locked RMWs, fences
        |
        |  the CPU preserves the ISA memory model, and may use
        |  a store buffer, out-of-order execution, and a coherent cache
        v
one core's retirement, then the coherence protocol
```

The first arrow is why a data race is so severe. Once the program leaves the model, the compiler's obligation to preserve the interleaving you imagined is gone. The second arrow is why a defined program can still show an outcome that the source lines, read top to bottom, do not suggest. Store buffers are the everyday example on x86-64: a store retires into a buffer before it becomes visible to other cores, and a later load of a different address can complete from the cache while that store is still buffered.

Kernel code separates the same two arrows with two different tools, both present in this tree.

`include/linux/compiler.h` defines the usual `barrier()` as an empty `asm volatile` with a `"memory"` clobber. That instruction emits no CPU fence. It tells the compiler that memory has been touched, so earlier loads and stores in the compiler's view stay earlier, and later ones stay later. `include/asm-generic/rwonce.h` defines `WRITE_ONCE` as a store through a volatile-qualified pointer of the object's type, after a compile-time size check. Volatile stops the compiler from deleting the store, merging it with a neighbor, or splitting it into a torn pair of smaller stores. It is still a compiler constraint. The CPU's memory model applies to whatever store instruction the compiler actually emits.

A CPU fence is an instruction in the architecture header: `mfence`, a `lock` prefix, `dmb`, `stlr`, `ldar`, `fence`. Those names show up in section 8. They are how this kernel asks a particular CPU to order its own accesses. They do not appear in the C source of the lab, and a C programmer asks for them by writing a memory order, then checks the assembly to see what this compiler did for this target.

Keeping the arrows apart is the point of the day's limit. An x86-64 process that prints one outcome has exercised the second arrow on one machine. The first arrow, the C abstract machine, is what makes the same source mean something on another compiler and another ISA. A litmus test that prints the expected order on x86-64 does not prove the C program.

5. Release and acquire

`memory_order_release` on a store and `memory_order_acquire` on a load are the orders that publish a payload. The pattern is:

```
writer                              reader
------                              ------
write payload bytes
atomic_store(&flag, 1, release)     if (atomic_load(&flag, acquire) == 1)
                                      read payload bytes
```

If the acquire load reads the value the release store wrote, or a later value in that object's modification order from the release sequence the standard defines, the release synchronizes with the acquire. Happens-before then orders the writer's earlier accesses before the reader's later accesses. The payload stores are visible to the payload loads. The flag is the message. The payload rides on the edge the message creates.

Several limits belong in the same picture.

The edge exists when the load reads that write. A reader that still sees the old flag has not synchronized, and it stays out of the payload. Spinning on the flag with acquire until the value appears is the usual way to wait for the edge. The queue in Day 4 does not use this pattern: it waits with a mutex and a condition variable, and the mutex unlock/lock pair is itself a release/acquire edge.

A release store orders *earlier* accesses in that thread against the store. It does not order later accesses in the writer against the store. An acquire load orders *later* accesses in the reader against the load. It does not reach backward into the reader's earlier accesses. An atomic read-modify-write that both publishes and observes uses `memory_order_acq_rel` when it must do both jobs.

Release/acquire is pairwise. Thread A releasing `flag1` and thread B acquiring `flag2` creates no edge. The litmus in section 7 is two relaxed operations on two different objects, so it has no release/acquire pair at all.

`memory_order_consume` was specified to order a load with later uses that depend on the loaded value, at a lower cost than acquire on some weakly ordered machines. In practice major compilers have strengthened it to acquire for years. The lab does not use it, and new code in this course should write acquire when a dependency is the intent.

6. Sequential consistency

`memory_order_seq_cst` places the operation in one total order, shared by every sequentially consistent operation in the program, consistent with the modification order of each object and with happens-before. A seq_cst store also has release semantics, and a seq_cst load also has acquire semantics. The extra promise is the single total order.

That promise is what makes the store-buffering outcome illegal for a seq_cst version of the litmus. Label the operations:

```
Thread 0:  Sx  store 1 to x, seq_cst
           Ly  load y, seq_cst
Thread 1:  Sy  store 1 to y, seq_cst
           Lx  load x, seq_cst
```

Sequenced-before puts `Sx` before `Ly` in thread 0 and `Sy` before `Lx` in thread 1, and the seq_cst total order respects that. Suppose both loads read 0, and the only stores in the round are the two stores of 1. Then `Ly` read 0, so `Sy` is not before `Ly` in the order that would have supplied the value 1. Likewise `Sx` is not before `Lx`. One consistent reading of the total order is the cycle:

```
Sx  before  Ly     (sequenced-before, both seq_cst)
Ly  before  Sy     (Ly observed the initial 0, so Sy is later)
Sy  before  Lx     (sequenced-before, both seq_cst)
Lx  before  Sx     (Lx observed the initial 0, so Sx is later)
```

A total order cannot contain a cycle. Both-zero is outside the seq_cst model. The lab does not compile this variant. The argument above is a reason to respect the order parameter, and it becomes a claim about a binary only after you inspect that binary's operations and the compiler's mapping. Seq_cst is also the default for the convenience functions that do not take an explicit order. The lab's checked paths pass an explicit order so the source states the promise in place.

Seq_cst is sufficient for a great many publication patterns and stricter than release/acquire. The counter does not need it. A single-flag message does not need it. The total order costs fences or locked instructions on machines where ordinary stores are cheap, which is visible in the headers in section 8: a full barrier is a different macro from a store-release.

7. Execution trace: the relaxed store-buffering litmus

`litmus_lab` runs 100000 rounds. `x` and `y` are `atomic_int`, set to 0 by the parent at the start of each round with relaxed stores. Two workers and the parent rendezvous on a pthread barrier. Worker 0 then does a relaxed store of 1 to `x` and a relaxed load of `y`. Worker 1 does a relaxed store of 1 to `y` and a relaxed load of `x`. A second barrier joins the round. The parent counts rounds in which both loads returned 0. The workers write those observations into separate slots of a plain `int observed[2]`. The finish barrier synchronizes those plain stores with the parent's reads, so the slots are not a data race. `x` and `y` are atomic, so the cross-thread accesses on them are not a data race either. The order on every atomic access in the round is relaxed.

Here is one execution the relaxed model allows. It is also the execution a store buffer produces on x86-64.

```
initial modification orders:  x: 0    y: 0

Thread 0                          Thread 1
--------                          --------
store x = 1
  enters store buffer
  not yet visible to thread 1
                                  store y = 1
                                    enters store buffer
                                    not yet visible to thread 0
load y
  cache still holds 0
  observed[0] = 0
                                  load x
                                    cache still holds 0
                                    observed[1] = 0
store buffer drains: x becomes 1, y becomes 1
parent's finish barrier, then both_zero increments
```

Nothing in this execution tears a store or invents a third value. Each object's modification order is still `0` then `1`. Each load read a real value from that order, the earlier one. There is no release store and no acquire load, so neither load synchronizes with the other thread's store. The pthread barriers order the *rounds*: the zeroing happens-before the workers start, and the workers' writes to `observed` happen-before the parent reads them. Inside the round they do not paste `x`'s modification order onto `y`'s.

On x86-64 the picture is close to the hardware. The architecture allows a load to pass an older store to a different address, which is what a store buffer does. The compiler can emit ordinary stores and loads for these relaxed atomics and the CPU can produce both-zero. On AArch64 and RISC-V the architecture allows still more reorderings of ordinary loads and stores, so the same source can produce both-zero for additional reasons. A run whose `both_zero` count is 0 is a report of this process, this compiler, this schedule, and these 100000 rounds. The printf in the lab records the count and then prints `caveat=x86_observation_is_not_C_language_proof`. Believe that line. The C abstract machine allows both-zero for `memory_order_relaxed` whether or not this CPU exhibited it today.

A seq_cst compilation of the same shape forbids the cycle in section 6. If you build that variant yourself, read the assembly before you interpret a zero count. A compiler that lowered seq_cst to the same plain stores as relaxed would not have implemented the model. The day's wrapper does not build that variant; it prints the relaxed counter's function so you can see a real lowering of a relaxed read-modify-write.

8. Architecture lens: three headers, three mappings

The kernel macros `smp_store_release`, `smp_load_acquire`, and `smp_mb` are the kernel's names for release stores, acquire loads, and full barriers between kernel memory accesses. `include/asm-generic/barrier.h` supplies a default. Each architecture header may define the `__smp_*` forms first and then include the generic file, which is what x86, AArch64, and RISC-V do in this tree. The generic default is a full barrier around a `WRITE_ONCE` or `READ_ONCE`:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: include/asm-generic/barrier.h
 * lines: 104–120
 * macros: __smp_store_release, __smp_load_acquire
 */
#ifndef __smp_store_release
#define __smp_store_release(p, v)                   \
do {                                        \
    compiletime_assert_atomic_type(*p);             \
    __smp_mb();                             \
    WRITE_ONCE(*p, v);                          \
} while (0)
#endif

#ifndef __smp_load_acquire
#define __smp_load_acquire(p)                       \
({                                      \
    __unqual_scalar_typeof(*p) ___p1 = READ_ONCE(*p);       \
    compiletime_assert_atomic_type(*p);             \
    __smp_mb();                             \
    (typeof(*p))___p1;                          \
})
#endif
```

An architecture that does not override these pays for a full `__smp_mb()` on every release store and every acquire load. The three architectures below do override them, and they do not override them the same way. Read the overrides as kernel engineering for a stated ISA, then keep C11 as a separate layer that a C compiler lowers on its own.

x86-64

`arch/x86/include/asm/barrier.h` in this tree:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: arch/x86/include/asm/barrier.h
 * lines: 57–79
 * macros: __smp_mb, __smp_store_release, __smp_load_acquire
 */
#ifdef CONFIG_X86_32
#define __smp_mb()  asm volatile("lock; addl $0,-4(%%esp)" ::: "memory", "cc")
#else
#define __smp_mb()  asm volatile("lock; addl $0,-4(%%rsp)" ::: "memory", "cc")
#endif
#define __smp_rmb() dma_rmb()
#define __smp_wmb() barrier()
#define __smp_store_mb(var, value) do { (void)xchg(&var, value); } while (0)

#define __smp_store_release(p, v)                   \
do {                                        \
    compiletime_assert_atomic_type(*p);             \
    barrier();                              \
    WRITE_ONCE(*p, v);                          \
} while (0)

#define __smp_load_acquire(p)                       \
({                                      \
    typeof(*p) ___p1 = READ_ONCE(*p);               \
    compiletime_assert_atomic_type(*p);             \
    barrier();                              \
    ___p1;                                  \
})
```

On the 64-bit path, `__smp_mb()` is a locked `addl` to a stack slot. The `lock` prefix makes that instruction a full barrier for ordinary memory, and the `"memory"` clobber is also a compiler barrier. `__smp_store_release` is not that instruction. It is `barrier()` followed by `WRITE_ONCE`: a compiler barrier, then a plain store. `__smp_load_acquire` is `READ_ONCE` followed by `barrier()`: a plain load, then a compiler barrier. Earlier in the same header, `dma_rmb()` is defined as `barrier()`, so `__smp_rmb()` is a compiler barrier too. `__smp_wmb()` is a compiler barrier directly.

The header does not contain the words "total store order." The mapping is still the one people summarize that way, and the summary should stay tied to the manuals rather than to a sentence this file does not have. For write-back memory, the Intel and AMD x86-64 models keep stores in program order with other stores, keep loads in program order with other loads, and keep a store from passing an older load. They allow a load to pass an older store to a different address. That last allowance is the store buffer in section 7.

Under those rules a store-release needs the compiler to keep the payload accesses before the flag store, and the CPU already keeps earlier stores and earlier loads before that store. A load-acquire needs the compiler to keep later accesses after the load, and the CPU already keeps later loads and later stores after that load. A compiler barrier plus a plain access is therefore the kernel's x86 mapping of release and acquire. A full barrier, the locked add, remains the mapping of `__smp_mb()`, which is what seq_cst and a few kernel protocols still call. This is a statement about the header you just read plus the published x86-64 model. It is not a promise that a given C compiler emits the same sequence for `memory_order_release`. Check the `.s` file when that is the claim.

AArch64

AArch64's ordinary loads and stores are weakly ordered. This tree does not implement store-release as a plain store. `arch/arm64/include/asm/barrier.h` emits a release store instruction. The same file sets `__smp_mb()` to `dmb(ish)`, `__smp_rmb()` to `dmb(ishld)`, and `__smp_wmb()` to `dmb(ishst)`: inner-shareable data memory barriers, full, load, and store.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: arch/arm64/include/asm/barrier.h
 * lines: 101–134
 * macro: __smp_store_release
 */
#define __smp_store_release(p, v)                   \
do {                                    \
    typeof(p) __p = (p);                        \
    union { __unqual_scalar_typeof(*p) __val; char __c[1]; } __u =  \
        { .__val = (__force __unqual_scalar_typeof(*p)) (v) };  \
    compiletime_assert_atomic_type(*p);             \
    kasan_check_write(__p, sizeof(*p));             \
    switch (sizeof(*p)) {                       \
    case 1:                             \
        asm volatile ("stlrb %w1, %0"               \
                : "=Q" (*__p)               \
                : "r" (*(__u8 *)__u.__c)        \
                : "memory");                \
        break;                          \
    case 2:                             \
        asm volatile ("stlrh %w1, %0"               \
                : "=Q" (*__p)               \
                : "r" (*(__u16 *)__u.__c)       \
                : "memory");                \
        break;                          \
    case 4:                             \
        asm volatile ("stlr %w1, %0"                \
                : "=Q" (*__p)               \
                : "r" (*(__u32 *)__u.__c)       \
                : "memory");                \
        break;                          \
    case 8:                             \
        asm volatile ("stlr %1, %0"             \
                : "=Q" (*__p)               \
                : "r" (*(__u64 *)__u.__c)       \
                : "memory");                \
        break;                          \
    }                               \
} while (0)
```

`stlrb`, `stlrh`, and `stlr` are AArch64 store-release instructions for 8, 16, 32, and 64 bits. The matching acquire loads are in `__smp_load_acquire` immediately below, lines 136–165 of the same file: `ldarb`, `ldarh`, and `ldar`, again selected by size, each with a `"memory"` clobber. Those mnemonics are the architecture's acquire loads. This lesson stops at naming the instructions the header emits. The Arm Architecture Reference Manual is the definition of how `STLR` and `LDAR` order against other accesses in an inner-shareable domain; the header is the evidence that this kernel uses them for `smp_store_release` and `smp_load_acquire`.

The contrast with x86 is the contrast the macros encode. x86's `__smp_store_release` is a compiler barrier and `WRITE_ONCE`. AArch64's is an explicit `stlr` family instruction. Both macros are called store-release in kernel code. The instructions inside them are the ISA showing through.

RISC-V

RISC-V's base memory model is weak, and this tree inserts a `fence` instruction rather than a plain store. `arch/riscv/include/asm/barrier.h`:

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: arch/riscv/include/asm/barrier.h
 * lines: 17–43
 * macros: RISCV_FENCE, __smp_mb, __smp_store_release, __smp_load_acquire
 */
#define RISCV_FENCE(p, s) \
    __asm__ __volatile__ ("fence " #p "," #s : : : "memory")

/* These barriers need to enforce ordering on both devices or memory. */
#define mb()        RISCV_FENCE(iorw,iorw)
#define rmb()       RISCV_FENCE(ir,ir)
#define wmb()       RISCV_FENCE(ow,ow)

/* These barriers do not need to enforce ordering on devices, just memory. */
#define __smp_mb()  RISCV_FENCE(rw,rw)
#define __smp_rmb() RISCV_FENCE(r,r)
#define __smp_wmb() RISCV_FENCE(w,w)

#define __smp_store_release(p, v)                   \
do {                                    \
    compiletime_assert_atomic_type(*p);             \
    RISCV_FENCE(rw,w);                      \
    WRITE_ONCE(*p, v);                      \
} while (0)

#define __smp_load_acquire(p)                       \
({                                  \
    typeof(*p) ___p1 = READ_ONCE(*p);               \
    compiletime_assert_atomic_type(*p);             \
    RISCV_FENCE(r,rw);                      \
    ___p1;                                  \
})
```

`fence pred,succ` orders the predecessor set against the successor set. `RISCV_FENCE(rw,w)` before the store orders earlier reads and writes before later writes, which is this tree's release fence. `RISCV_FENCE(r,rw)` after the load orders the load, a read, before later reads and writes, which is this tree's acquire fence. `__smp_mb()` is `fence rw,rw`, a full memory fence, distinct from `mb()`, which is `fence iorw,iorw` and also orders I/O. The `"memory"` clobber on the fence is again the compiler half.

A comment just below this excerpt, on `smp_mb__after_spinlock`, says the kernel's own lock sequence uses acquire/release bits on the atomic instructions and that those bits give an RCpc critical section on that one lock. The comment's conclusion in this tree is to emit a full `fence iorw,iorw` after the lock anyway. The detail that matters here is the caution already present in

the source: a RISC-V acquire/release pair in this kernel is not treated as a drop-in sequential-consistency fence. When you map a C11 order onto RISC-V, cite the compiler output and the ISA manual for that order, and expect a `fence` or an `aq`/`rl` bit where x86 had a plain `mov`.

What these three mappings do not decide

They do not decide the assembly of `increment_atomic` in the lab. That function is a C11 `atomic_fetch_add_explicit` with `memory_order_relaxed` on an `atomic_uint_fast64_t`. A relaxed read-modify-write still has to be atomic. On x86-64 the usual instruction is a `lock`-prefixed add or `xadd`, because atomicity of a read-modify-write is a locked bus operation even when the memory order is relaxed. Confirm the opcode in the file the wrapper writes. A locked add is a full barrier on x86 as a *consequence of the instruction the CPU requires for atomicity*. That hardware consequence is stronger than the C order the source requested. Other threads still must not rely on it for portable publication. On AArch64 the same C function may be an exclusive loop or an LSE atomic, depending on the target flags, and the relaxed form need not be `stlr`/`ldar`. On RISC-V it may be an AMO with or without `aq`/`rl`. This tree's kernel headers show the kernel's choices. The wrapper's `.s` file shows one C compiler's choice for one function on the host you actually run.

9. How to read the evidence without promoting it

The wrapper's prediction line is the right size. The atomic count is exact: 800000. The litmus prints `both_zero` and does not require a particular value. The assembly path is best-effort; a failure to generate it is a skip recorded in `assembly.log`, and the counter result stands on its own.

When the `.s` file exists, find `increment_atomic` and name the instruction that updates memory, including any `lock` prefix or fence. Record `cc --version` and `uname -m` from the wrapper's output beside that instruction. The pair is "this compiler, this target, this function." It becomes a portability claim only after the same source is lowered for another target, and even then the claim is about those compilers, not about every future one.

Leave these as open, because the lab does not close them:

- every interleaving the abstract machine allows;
- the seq_cst lowering, which the wrapper does not generate;
- glibc's internal atomics inside a mutex or a condvar;
- the kernel's `smp_store_release` as if it were the C library.

Day 4 already showed `set_current_state` storing task state with `smp_store_mb`, and `futex_wait_setup` depending on the hash-bucket lock to keep a wake from passing the waiter's publication. Those are kernel clients of the barriers in this section. The queue lab remains a mutex program. It does not become a lock-free program because the kernel uses atomics on the sleep path.

10. Mastery: Explain · Draw · Observe · Build

Explain

- What a modification order is, and why eight hundred thousand relaxed fetch-adds on one counter have a determined final value.

- Which extra edge the parent's `pthread_join` supplies before the load.
- What a release store synchronizes with, and which payload accesses ride on that edge.
- Why the seq_cst both-zero cycle cannot be a total order, and why the relaxed litmus may print both-zero anyway.
- Which instruction this tree's x86 header uses for `__smp_store_release`, which instruction the AArch64 header uses, and which fence the RISC-V header places before the store. Say, in the same answer, that those are kernel macros.

Draw

Draw the two arrows of section 4 for the litmus: source operations, then the instructions you think this host emits, then the store buffers and the loads that return 0. Draw the modification orders of `x` and `y` as two separate chains. Draw the seq_cst cycle underneath and mark it as a forbidden total order. The relaxed binary is allowed to print both-zero.

Observe

Run challenges/day-005-atomics-memory-order.sh. Keep the exact counter line, the litmus line including `both_zero` and the caveat, the compiler version, `uname -m`, and either the `increment_atomic` assembly or the skip line. If `both_zero` is 0, write the sentence from section 7 about what that count measures.

Build

From the printed assembly and the three headers, make a three-row note: x86-64, AArch64, RISC-V. Each row names the kernel store-release sequence quoted above and states whether a CPU fence instruction is present in that macro. Add a fourth row for the lab's relaxed fetch-add, filled only with what the wrapper's assembly file shows on this host. The note is done when a reader can see that a plain x86 store-release in the kernel header, an `stlr` on AArch64, a RISC-V `fence rw,w`, and a locked counter increment in a C binary are four different artifacts.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Atomics Memory Order

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-005-atomics-memory-order.sh
```

- View lab source on GitHub

Day 6 — Liveness, False Sharing, and Scalability

Target time: approximately 3 hours

> A protocol can keep its invariant on every step and still fail to finish. Deadlock is a cycle of threads each waiting for a lock another thread in the cycle holds. False sharing is a different failure of progress: the invariant holds, the atomics are race-free, and the cores still spend the interval moving one cache line back and forth.

Today's wrapper runs both phenomena and keeps their evidence different. challenges/day-006-liveness-scalability.sh expects the timed cycle detector to exit and the globally ordered mode to finish. It prints three timing samples of the adjacent and padded counters and requires no winner. A measurement from those samples describes one machine, one compiler, and one layout.

1. Safety and liveness answer different questions

Safety for the mutex counter on Day 3 is a statement about states: every time a thread reads or writes `mutex_counter`, it holds `counter_lock`, so the counter moves by one published increment at a time. Liveness is a statement about futures: a thread that needs the lock eventually acquires it, does the increment, and releases it. The ordered two-lock mode in today's lab is a small liveness policy. The detector mode is a small demonstration that the opposite policy waits in a cycle, with a timeout so the process still terminates.

The C abstract machine will not detect either outcome for you. A deadlock of well-locked mutexes is defined behavior that makes no progress. A data-race-free atomic program that false-shares is defined behavior that makes progress slowly. Day 2 removes programs from the model when non-atomic accesses conflict. It does not rank the defined programs by how long they take, and it does not require a scheduler to be fair.

Hardware sits under both. A thread blocked in a futex wait owns no core: its userspace registers were saved at the syscall boundary, its stack stays mapped in the shared address space, and `schedule` runs somebody else. A thread spinning on an atomic in userspace owns a core and a cache line. The deadlock lab uses the first shape, bounded by a timeout. The false-sharing lab uses the second. Mixing them up produces the wrong repair: adding a timeout does not separate two counters onto two lines, and padding a struct does not break a lock cycle.

2. Deadlock is a wait-for cycle

Draw the waits. The order of the source lines only suggests which edges can appear.

```
Thread A holds lock 0 and waits for lock 1
Thread B holds lock 1 and waits for lock 0

       owns              waits for
   A ---------- lock 0
   |                        |
   |                        v
   +---- waits for ---- lock 1 ---------- B
```

Each arrow is a thread that will not release what it holds until it acquires what it waits for. The graph has a cycle. No thread in the cycle can run the unlock that the next thread needs. That is deadlock.

Four ingredients show up together in this kind of cycle, and naming them makes the repair obvious:

1. The locks are exclusive. One owner at a time.
2. A thread holds one lock while it waits for another. That is hold-and-wait.
3. Nothing steals a lock from an owner. The owner runs the unlock, or the wait gives up.
4. The waits form a cycle.

The lab's ordered mode leaves 1–3 in place and removes 4. Every path that takes both locks takes `locks[0]` before `locks[1]`. A thread that holds `locks[1]` does not go on to wait for `locks[0]`, because it already acquired `locks[0]` earlier in the same path and will release `locks[1]` first. The wait-for graph cannot contain the edge "holds 1, waits for 0."

The detector mode builds the cycle on purpose and then refuses to wait forever. `pthread_mutex_timedlock` returns `ETIMEDOUT` when the deadline passes. The thread unlocks the mutex it still holds. The process joins both threads and exits. `test_hung=no` is the claim the program is willing to print: this run ended. It is not a claim that a non-timed version of the same lock order would have ended.

3. Execution trace: the timed cycle

`deadlock_lab detect` creates two threads with `struct attempt` values `{first: 0}` and `{first: 1}`. Each thread locks `locks[first]`, meets the other at a rendezvous, then timed-locks `locks[1 - first]` with a deadline 100 milliseconds after `CLOCK_REALTIME`. The rendezvous is a third mutex, `gate_lock`, and a condition variable. Its only job is to make both "first" acquisitions visible before either "second" attempt. Without it, one thread might acquire both resource locks before the other thread starts, and the run would show no cycle.

Start with `locks[0]` and `locks[1]` free, `gate_count` 0, both threads runnable.

```
step  who   holds                         waits               gate_count
----  ----  ----------------------------  ------------------  ----------
 1    T0    locks[0]                      —                   0
 2    T1    locks[1]                      —                   0
 3    T0    locks[0], then gate_lock      —                   1
 4    T0    locks[0]                      gate_cond           1
      (cond_wait released gate_lock)
 5    T1    locks[1], gate_lock           —                   2
      T1 broadcasts, unlocks gate_lock
 6    T0    locks[0], reacquires gate     —                   2
      T0 sees gate_count == 2, unlocks gate_lock
 7    T1    locks[1]                      —                   2
      T1 leaves the rendezvous
 8    T0    locks[0]                      locks[1] (timed)    2
 9    T1    locks[1]                      locks[0] (timed)    2
10    one or both timedlocks return ETIMEDOUT
11    each thread unlocks the lock it still holds
12    parent joins both and prints test_hung=no
```

Steps 1 and 2 are the hold. Steps 8 and 9 are the cycle. Between them the gate does something that looks like a wait and is not the deadlock. At step 4, T0 holds `locks[0]` and waits on `gate_cond`. That wait is a Day 4 wait: `pthread_cond_wait` releases `gate_lock`, so T1 can acquire it, raise `gate_count` to 2, and broadcast. The predicate `gate_count == 2` becomes true, both threads recheck it under `gate_lock`, and both release `gate_lock` before they touch the second resource lock. The gate's job is to publish "both first locks are held." Once that predicate is true, the gate is no longer in the wait-for graph.

At step 8, T0's `pthread_mutex_timedlock` on `locks[1]` finds T1's ownership and blocks. glibc's usual implementation waits through a futex, the same kernel sleep sketched on Day 4, with a timeout attached. T0's userspace registers are saved. The core is free to run T1. At step 9, T1 tries `locks[0]`, which T0 still owns, and blocks the same way. Now neither thread is running the unlock the other needs. The cycle is real, and it is inside the kernel's wait queues rather than in a userspace spin.

The deadline each thread computed is `CLOCK_REALTIME` plus 100000000 nanoseconds, with the nanosecond field normalized if it overflows. When that time is reached, the futex wait returns and `pthread_mutex_timedlock` returns `ETIMEDOUT`. The lab treats success as "at least one of the two results is `ETIMEDOUT`." Both may time out. The printed integers are those pthread return codes. After the return, a thread that happened to acquire the second lock unlocks it, and every thread unlocks its first lock. The parent joins. The process is gone. `deadlock_cycle_detected=yes` together with `test_hung=no` means the cycle was observed through a bounded failure and the cleanup ran.

What the trace does not include is a rollback. The lab's critical sections contain no payload, so there is nothing to undo. A timed lock in a program that had already published a field under the first mutex would return with that field still published. The timeout ends the wait. Repair of partial work is a separate protocol, and this source does not have one.

4. One global order removes the cycle

`deadlock_lab ordered` starts two threads that both run the same sequence: lock `locks[0]`, lock `locks[1]`, unlock `locks[1]`, unlock `locks[0]`. There is no rendezvous and no timed call. One thread gets `locks[0]` first. The other blocks on `locks[0]` holding nothing. The owner

takes `locks[1]`, releases it, releases `locks[0]`, and the waiter then repeats the same order. The wait-for graph is a single edge from the waiter to `locks[0]`, and it disappears when the owner unlocks.

```
wrong for a second path:          right for every path:

  sometimes 0 then 1                always 0 then 1
  sometimes 1 then 0
        \                          lock 0 -----> lock 1 -----> unlock 1
         \                                                      |
          +-- cycle appears                unlock 0 --+
              when the two orders meet
```

The rule has to be global to be a rule. "Usually 0 before 1" leaves one error path, one callback, or one logging helper that takes them backward, and that path is enough to rebuild the cycle. The lab is small enough that both call sites are visible in `mode_ordered` and `opposite_order`. A larger program writes the order down and holds to it on the failure returns as well as on the happy path. Day 3 stated that obligation; today the detector shows the cycle the obligation exists to prevent.

Try-lock and timed-lock are escape hatches with their own postcondition. The detector uses a timeout as a sensor. A program that uses `timedlock` as a protocol has to say what it does with `ETIMEDOUT`: retry the whole ordered sequence from the beginning, drop the work, or take a coarser lock. Returning into the caller while still holding `locks[0]` and pretending the second update happened would break the invariant the two locks were protecting. The detector unlocks and exits, which matches a sensor and would be incomplete as a transaction.

5. Starvation is a different graph

Starvation is a thread that stays runnable, or stays blocked on a lock that other threads keep acquiring, for as long as you care to watch, while the system as a whole continues. There is no cycle. There is a schedule that never picks the unlucky thread.

POSIX mutexes do not promise FIFO ownership. A scheduler does not promise that a runnable thread runs within a bound this lab could check. The ordered mode happens to give each of two threads one turn and then joins them, so a passing run has no room to display starvation. A passing run is compatible with a mutex implementation that, under a different loop, would hand `locks[0]` to the same thread every time.

The detector's timeout is also the wrong instrument for starvation. A timeout fires because time passed, including when the lock was busy for a good reason and the holder was making progress. `ETIMEDOUT` in the detector is informative because the rendezvous has already established that the other thread holds the contested lock and is itself waiting. The cycle is the explanation. The clock only bounded the wait. On a loaded machine the same clock can expire for a lock whose owner is merely slow. The lab does not try to tell those apart with a second experiment; it forces the cycle first.

Fairness, priority inheritance, and robust mutexes after a dead owner are real pthread features and they are outside this wrapper. The wrapper's limit line says so: no fairness proof, no starvation proof.

6. False sharing starts from the cache line

Week 2's cache organization fixed the unit the memory hierarchy actually moves. A core that needs one address fills a whole line, commonly 64 bytes on current x86-64 parts, with the neighboring bytes that share that line's base. The line is the unit of coherence as well as the unit of fill. Two cores may each hold a copy. A write needs a writable exclusive copy, and the other copies are invalidated or downgraded. Week 2's name for the resulting traffic, when the two cores are writing *different* objects that happen to sit on one line, is false sharing.

The lab builds that situation with atomics so the C program stays defined.

```
struct adjacent {
    atomic_uint_fast64_t value[2];   /* two objects, one likely line */
};

struct padded_counter {
    _Alignas(64) atomic_uint_fast64_t value;
    unsigned char padding[64];
};
```

`mode_false_sharing` runs two threads. Each thread calls `atomic_fetch_add_explicit` 200000 times on its own counter, with `memory_order_relaxed`. One run uses `&near.value[0]` and `&near.value[1]`. The next uses `&far[0].value` and `&far[1].value`. Day 5 already said why the final values would be exact if the lab checked them: each counter has its own modification order. The lab does not check them. It prints wall-clock seconds and the caveat `timings_are_noisy_and_not_a_correctness_test`.

The addresses are virtual. The cache, on the machines Week 2 described, tags physical lines; L1 is often virtually indexed and physically tagged, and the exact split is microarchitectural. Inside one page, a normal anonymous mapping is contiguous in physical memory, so a virtual gap of one or more line sizes is a physical gap of the same size. `_Alignas(64)` asks the compiler to place `value` on a 64-byte boundary. The trailing `padding[64]` plus the struct's alignment make `sizeof(struct padded_counter)` a multiple of 64, so `far[0]` and `far[1]` land in different 64-byte blocks. `near.value[0]` and `near.value[1]` are adjacent `atomic_uint_fast64_t` objects, typically 8 bytes each, and those 16 bytes share a 64-byte line whenever the array itself is not split across a line boundary by a perverse placement. The wrapper never prints the addresses, the sizes, or `getconf LEVEL1_DCACHE_LINESIZE`. A timing gap is consistent with that layout on a machine whose line size is 64. It is not a measurement of the line size, and a line size of 128 would change which of these gaps is enough.

```
one 64-byte line, adjacent case

| value[0] (thread 0 writes) | value[1] (thread 1 writes) | 48 bytes unused |
+----------------------------+----------------------------+-----------------+
  both fetch-adds are RMWs of this same coherence unit

padded case

| far[0].value | pad ........ | far[1].value | pad ........ |
+--------------+--------------+--------------+--------------+
  line A, thread 0 only        line B, thread 1 only
```

The unused bytes in the adjacent line are the expensive part. The hardware has no field in the line that says "thread 0 promised to touch only the first eight bytes." Coherence grants and revokes the line.

7. The MESI picture Week 2 already implied

Week 2's lesson did not need the four state names to make the traffic real. The names are the compact picture of what it did say: a writable copy is exclusive, other copies lose write permission, and a later reader has to fetch the line again. MESI is that picture.

```
state        this cache's copy                          a write from here
-----------  -----------------------------------------  ------------------------
Modified     only copy, dirty relative to memory      already exclusive; write hits
Exclusive    only copy, clean                          write hits, state becomes Modified
Shared       other caches may also hold a clean copy  must invalidate the others first
Invalid      no copy                                    must obtain the line first
```

A false-sharing bounce between two cores, each doing a relaxed fetch-add to its own half of one line, is a walk around those states:

```
Core 0                        Core 1                      the line
------                        ------                      --------
fetch-add value[0]
  line arrives Exclusive
  RMW writes, line is Modified
                              fetch-add value[1]
                                needs the line writable
                                Core 0's copy → Invalid
                                Core 1's copy → Modified
fetch-add value[0] again
  Core 1's copy → Invalid
  Core 0's copy → Modified
```

Each step's arithmetic is a handful of cycles once the operand is in a register. The miss that fills the register waits on the interconnect, on the other core's cache, and possibly on a snoop filter or a last-level cache. The C abstract machine does not contain those cycles. The ISA guarantees that the locked or atomic read-modify-write observes a single modification order for that object. How long the line took to arrive is a property of this die, this topology, and this placement.

Real cores extend MESI. Intel parts are often described with a Forward state (MESIF). AMD parts are often described with an Owned state (MOESI). A directory or a snoop filter can answer "who has the line?" without a bus broadcast. None of that changes the lab's qualitative fact: the coherence granule is the line, so two writers on one line invalidate each other, and two writers on two lines do not. Naming MESI here is naming Week 2's exclusive-ownership rule. It is not a claim that the CPU under this particular run implements exactly four states and no directory.

The fetch-add also touches the core's own machinery. The atomic value has to be in a register for the add. On x86-64 a `lock`-prefixed add holds the line in a writable state for the instruction and is a full barrier as a side effect of the `lock` prefix, which Day 5 already separated from the C order `memory_order_relaxed`. The false-sharing cost is paid on top of that atomicity cost. A single thread doing both counters would pay the atomicity cost and would not bounce the line

between cores. The lab always uses two threads, so it does not isolate those two costs. The three samples remain a comparison of two layouts under one harness, not a decomposition into "cycles of `lock add`" versus "cycles of snoop."

8. How this kernel asks for a private line

The kernel fights the same placement problem with alignment attributes. Tree `c9b2385d6793`, `include/linux/cache.h`, states the intent for read-mostly data and then defines the SMP alignment macros. `____cacheline_aligned_in_smp` is the one to remember: under `CONFIG_SMP` it is an alignment to `SMP_CACHE_BYTES`; on a uniprocessor build the macro is empty, because there is no second cache to bounce a line with.

```
/* tree c9b2385d6793 (Linux 5.10.134)
 * path: include/linux/cache.h
 * lines: 16–50
 * macros: __read_mostly, ____cacheline_aligned_in_smp
 */
/*
 * __read_mostly is used to keep rarely changing variables out of frequently
 * updated cachelines. Its use should be reserved for data that is used
 * frequently in hot paths. Performance traces can help decide when to use
 * this. You want __read_mostly data to be tightly packed, so that in the
 * best case multiple frequently read variables for a hot path will be next
 * to each other in order to reduce the number of cachelines needed to
 * execute a critical path. We should be mindful and selective of its use.
 * ie: if you're going to use it please supply a *good* justification in your
 * commit log
 */
#ifndef __read_mostly
#define __read_mostly
#endif

/*
 * __ro_after_init is used to mark things that are read-only after init (i.e.
 * after mark_rodata_ro() has been called). These are effectively read-only,
 * but may get written to during init, so can't live in .rodata (via "const").
 */
#ifndef __ro_after_init
#define __ro_after_init __section(".data..ro_after_init")
#endif

#ifndef ____cacheline_aligned
#define ____cacheline_aligned __attribute__((__aligned__(SMP_CACHE_BYTES)))
#endif

#ifndef ____cacheline_aligned_in_smp
#ifdef CONFIG_SMP
#define ____cacheline_aligned_in_smp ____cacheline_aligned
#else
#define ____cacheline_aligned_in_smp
#endif /* CONFIG_SMP */
#endif
```

`SMP_CACHE_BYTES` defaults in the same header to `L1_CACHE_BYTES` when an architecture has not overridden it. That is a compile-time constant of the kernel build, not a value the userspace lab queries. The comment on `__read_mostly` is the dual of false sharing: pack variables that are read together onto as few lines as possible, and keep them off the lines that

writers hammer. Alignment to a cache line does the other job, giving a hot counter a line of its own. The comment also says to justify the attribute. Padding is not free. A line reserved for one counter is a line the rest of the working set cannot use, and a structure that used to fit in one cache fill now takes two.

The lab's `_Alignas(64)` is a userspace request of the same shape, with the line size written as a literal. It is not the kernel macro, and it is not consulted from `SMP_CACHE_BYTES`. If you compare them, compare the idea — put independently written objects on different coherence granules — and keep the two source texts in their own projects.

9. Adding cores helps independent work

Scalability is the shape of the work, drawn against the number of cores that can run it.

```
independent counters, private lines
  core 0 writes line A
  core 1 writes line B
  core 2 writes line C
  each RMW completes from the local cache after the first fill
  more cores, more completed increments, until bandwidth or the
  serial fraction of the program runs out

one shared line
  every writer invalidates every other writer's copy
  the line occupies the interconnect in proportion to the writes
  another core is another contender for the same granule
```

Amdahl's observation fits the first shape when you can name the serial fraction. If a portion `s` of the runtime must happen on one core, and the rest is parallel, then even an infinite number of cores leaves you with that portion `s`. The shared-line program is harsher than a plain serial fraction: the contended writes actively slow down as contenders are added, because each write waits for exclusive ownership of a line that the new cores are also taking. Adding cores helps the private-line work. Adding cores increases the coherence traffic of the shared-line work.

The lab never varies the core count. It runs two threads, twice, and times each pair with `CLOCK_MONOTONIC` around `pthread_create`, the increments, and `pthread_join`. Thread creation and join are inside the number you read. The scheduler may place both threads on one core, on two cores, or migrate them mid-run. The wrapper asks `getconf _NPROCESSORS_ONLN` and `uname` so the report has an identity, and it does not pin threads with `taskset`. Three samples are what the script prints. They will not agree with each other down to the microsecond, and the script does not require `padded_seconds` to be smaller than `adjacent_seconds`.

A microbenchmark measures one machine, one compiler, one layout, one iteration count, and one timer placement. A result that matches the MESI story on this host is support for the story here. A result that does not is a fact about this run: one core, a noisy timer, a line size other than 64, a hypervisor that multiplexes the two threads onto one physical core, or a `lock`-prefixed instruction whose local cost dominates the bounce. Record the numbers and the identity. Keep the correctness claim where it already is, in the atomic modification order and the deadlock markers, not in which sample was faster.

Virtual memory and the scheduler belong in that record. Migration moves a thread onto a core whose private cache does not hold the line, so a correctly padded counter still misses after a migration. A page that the kernel places far away in a NUMA machine pays a remote latency even when the line is uncontended. Week 2's hierarchy — TLB, private caches, shared last level, memory controller — is the path each fetch-add walks. Today's lab times the whole path plus thread lifetime, and it stops there.

10. Mastery: Explain · Draw · Observe · Build

Explain

- Draw, in words, the wait-for cycle at step 8 of the trace, and say why the gate's condition wait is a finished rendezvous by then.
- Why `pthread_mutex_timedlock` lets the process print `test_hung=no`, and what state a program would still have to repair if the first lock had protected a real update.
- Why the ordered mode's graph has no cycle as long as every path takes index 0 before index 1.
- Why two relaxed atomics on one line produce coherence traffic, and why that traffic is not a C data race.
- What `____cacheline_aligned_in_smp` expands to in an SMP build of this tree, and why the userspace lab's `_Alignas(64)` is a related idea with a hard-coded size.

Draw

Draw the two-lock cycle with the gate already released. Draw a 64-byte line containing `value[0]` and `value[1]`, then the same bytes after padding, and mark MESI state on each line for three successive fetch-adds. Use Week 2's vocabulary for anything you are unsure the four letters capture: exclusive ownership, invalidation, line rather than object.

Observe

Run challenges/day-006-liveness-scalability.sh. Keep `uname`, the online processor count, both deadlock markers, the three timing samples, and the source the wrapper prints for the cycle and the ordered path. Write one sentence on whether the three padded samples agree with each other. That sentence is part of the evidence.

Build

Write two short accounts next to the wrapper output. The first is the wait-for graph of `detect` at the moment both timed locks are in flight, and the graph of `ordered` when one thread holds both locks. The second is a statement of what the three samples are allowed to decide: one host, one compiler, one layout, timers that include create and join, no affinity, no hardware counters, no required winner. End with the scale picture from section 9: private lines gain from more cores; a shared written line spends those cores on coherence.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Liveness Scalability

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-006-liveness-scalability.sh
```

- View lab source on GitHub

Day 7 — Debuggers, Sanitizers, and Three Systems

Target time: approximately 3 hours

> Each tool stops the program at a different layer and answers a different question. A report is finished when it says which layer produced each fact, which binary those facts belong to, and which questions that layer leaves open.

The day's wrapper builds that report for you and leaves the interpretation as the work. challenges/day-007-debug-report.sh writes a temporary file, runs the deterministic Week 11 suite, then runs the intentional race, the timed deadlock, the false-sharing samples, and the litmus, then tries GDB, Valgrind Memcheck, ASan/UBSan, and ThreadSanitizer. Predict `WEEK11 PASS` from the suite. Predict that the deadlock detector exits. Treat every optional tool as "evidence, or an accurate skip."

1. Why the layers have to stay apart

A concurrent C program is several machines at once.

```
C abstract machine
  objects, lifetimes, happens-before, data-race rules
        |
        | compiler may instrument (ASan, UBSan, TSan)
        v
userspace binary
  registers, instructions, heap allocator
        |
        | Valgrind translates this binary as it runs
        | GDB stops it and reads architectural state
        v
kernel tasks
  clone, stacks, futex sleep, ptrace, the scheduler
        |
        v
caches, store buffers, the coherence protocol
```

A fact established on one arrow does not automatically establish the next. GDB can show that two threads exist and that one of them is inside `pthread_cond_wait`. That is a snapshot of tasks and registers. It does not say whether the `while` predicate in the source is the right one. ThreadSanitizer can report a conflicting pair of accesses. That is a fact about an instrumented schedule. It does not print the cache line those addresses share, and a run that prints nothing has not surveyed the abstract machine. Valgrind Memcheck can say the queue's result objects were freed. That is a fact about the allocator's bookkeeping on that run. It is silent about modification order.

The intentional data race belongs only on the paths that name it. The deterministic suite, `scripts/test-week11.sh` in the lab, runs the checked modes and prints `INFO race mode is intentionally excluded`. Day 2's wrapper and today's wrapper are the places that invoke `thread_lab race`. That mode increments a plain `uint64_t` from four threads. Day 2 already

classified it: conflicting non-atomic accesses, no happens-before, undefined behavior. The wrapper runs it as a bounded specimen. A printed count, including a count that equals 800000, is part of the specimen. It is not a test that the source is defined.

2. Identity before interpretation

The report starts with a UTC timestamp, `uname -a`, the first line of `cc --version`, and SHA-256 hashes of `thread_lab`, `queue_lab`, `deadlock_lab`, and `litmus_lab`. Those hashes name the binaries the later sections execute. A sanitizer rebuild is a different binary, compiled with different flags, and the report keeps it in its own section so a stack trace from that binary is not matched against the hashed uninstrumented file.

Concurrency bugs move when the binary changes. A log line added to "see the race" changes timing, register allocation, and sometimes whether the compiler is still willing to keep a value in a register. Capture the report from the binary you intend to talk about, and if you edit the source, say so and hash again. Optimization level matters for the same reason: the ASan build in the wrapper uses `-O1`, the ordinary lab build uses the Makefile's flags, and Day 5's assembly sample uses `-O2`. Three translations, three identities.

The kernel that executes the lab is whatever `uname` prints. The kernel *text* quoted on Days 4, 5, and 6 was read from tree `c9b2385d6793`, whose top-level `Makefile` says version 5.10.134. Those are allowed to differ. A futex path you traced with `strace` on the host is the running kernel's path. A comment quoted from `kernel/futex.c` is the tree's comment. Version discipline is writing both names down.

3. Execution trace: one report, top to bottom

What follows is the trace of the wrapper, in the order the sections appear. It records evidence. The interleaving of the four worker threads is a separate record, and this wrapper does not claim to have captured it.

```
section            program / tool              what a careful reader may conclude
-----------------  --------------------------  ----------------------------------
identity           uname, cc, sha256           which host, compiler, and binaries
deterministic      make test-week11            the checked modes printed their
                                               markers, including WEEK11 PASS,
                                               and the race mode was not among them
race               thread_lab race             the UB specimen ran and printed
                                               some count; the count is not a spec
deadlock           deadlock_lab detect         the timed cycle returned; test_hung=no
false-sharing      thread_lab false-sharing    one pair of wall-clock samples
litmus             litmus_lab                  both_zero for this many rounds,
                                               plus the x86-is-not-C-proof caveat
gdb                gdb batch, mode basic       threads and a completed join,
                                               or a skip naming ptrace/symbols
valgrind           memcheck, queue_lab         leak check of that run, or a skip
asan-ubsan         instrumented ownership      that path's shadow checks, or a skip
tsan               instrumented race           a data-race report, a runtime skip,
                                               or a skip that withholds any pass
landmarks          three printed lines         a reminder to pin ToyOS, a teaching
                                               OS revision, and the Linux tree
limits             one printed sentence        no exhaustive schedule, no fairness
                                               proof, no portability from one ISA
```

Read the trace as a stack of scopes. `WEEK11 PASS` means the patterns in `test-week11.sh` matched: create/join, the mutex counter 800000, the atomic counter 800000, ownership 42, the queue's while-predicate marker, the deadlock detector's `test_hung=no`, the ordered lock mode, the false-sharing caveat string, and the litmus caveat string. Several of those patterns are checks that a caveat was printed, which is a check that the program admitted its limit. They are not checks that the padded counter was faster or that `both_zero` had a required value.

4. GDB reads architectural state at a stop

The wrapper runs, when `gdb` is on `PATH`:

```
gdb -batch -ex 'break mode_basic' -ex 'run basic' -ex 'info threads'
    -ex 'continue'
```

`mode_basic` creates one thread, joins it, and checks that the pointer came back. The breakpoint stops the process at that function. `info threads` lists the threads GDB can see at the stop. `continue` lets the process run to exit, and the wrapper looks for `pthread_create_join=ok` in the log.

Underneath, GDB uses `ptrace`. The kernel marks the task traced, arranges a stop at the breakpoint (a patched instruction, or a hardware debug register, depending on the breakpoint), and hands GDB the saved registers. At the stop you can ask for a backtrace per thread, the instruction pointer, the stack pointer, and, with debug symbols and a cooperative glibc, the mutex owner words and condition-wait frames. Those are the architectural and ABI state of a paused process. They are the same registers Week 2 taught you to track, frozen at one moment.

The pause is part of the data. A thread that was runnable is now stopped. Wakeups that would have raced it do not race it while the process is stopped. A deadlock investigation uses this on purpose: stop everyone, record for each thread whether it is running, blocked in a futex, or sitting in userspace, and draw the wait-for graph from Day 6. A single stop that shows no cycle shows that the cycle was not present at that stop. Other schedules remain possible. The wrapper's batch session is narrower still. It confirms that GDB could stop `mode_basic`, list threads, and reach the success line. It does not walk a deadlocked process. The deadlock evidence in the same report is the timed-lock return code. This breakpoint does not capture a deadlocked backtrace.

If `gdb` is missing, or the batch command fails, the wrapper prints a skip and keeps the diagnostic log. A ptrace policy that rejects the tracer is a property of the environment. The skip records that no stop-state evidence was collected. The create/join result from the uninstrumented binary is a different section and still stands.

5. Valgrind Memcheck follows heap ownership

When `valgrind` is installed, the wrapper runs Memcheck on `queue_lab` with `--leak-check=full`, `--errors-for-leak-kinds=all`, and an error exit code of 91. Memcheck does not run the binary on the raw CPU. It translates basic blocks as the process executes and keeps shadow state beside the program's memory: which bits have defined values, and which heap blocks the allocator handed out and accepted back.

That is the right tool for the queue's result objects. Each consumer `calloc`s a `struct result`, returns it through `pthread_join`, and the parent `free`s it. A clean Memcheck run supports "on this execution, those blocks were allocated, used, and released, and no leak Memcheck tracks was left behind." The queue's ring itself is a static object with static mutexes and condition variables; Memcheck is not being asked to prove the ring invariant. The ring invariant is the source protocol from Day 4, checked by the sum and the final `count`.

Memcheck's shadow state is not a happens-before relation. Valgrind's race detectors are Helgrind and DRD, and the wrapper does not invoke them. A clean Memcheck log of `queue_lab` is evidence about allocation lifetime on that run. The data-race specimen is a different program section, and Memcheck is not the tool the wrapper points at it.

If Valgrind is absent, the skip is the evidence. If it exits 91, the wrapper keeps the log as `FAIL-EVIDENCE` and does not relabel that as success. Read the log against the source before changing anything: a leak in the result path would be a lifetime bug, and a complaint inside the C library's thread startup would be a tool-and-runtime interaction to record as such.

6. ASan and UBSan instrument the translation

The wrapper compiles `thread_lab.c` with `-fsanitize=address,undefined`, `-fno-omit-frame-pointer`, and `-O1`, then runs the `ownership` mode with `ASAN_OPTIONS=detect_leaks=1`.

AddressSanitizer is a compiler pass plus a runtime. The pass inserts checks around loads and stores and asks the allocator for redzones around heap objects. The runtime keeps a shadow map from addresses to "accessible / redzone / freed." A load or store whose shadow says the address is poisoned aborts with a stack trace. Leak detection at exit walks the heap for blocks with no remaining pointer. The ownership mode is a single allocation, a handoff through `pthread_create`, a write by the worker, a join, and a free by the parent. A clean run supports that path under this instrumentation. It does not run `race`, `false-sharing`, or the queue.

UndefinedBehaviorSanitizer, enabled by the same `-fsanitize=undefined`, inserts the checks the compiler version knows how to insert: typical sets cover null references, out-of-bounds shifts, signed overflow in the patterns the pass handles, and a list that changes with the compiler. A data race is undefined behavior in the C abstract machine and is not what this UBSan build is being asked to catch. The race specimen is reserved for the TSan section. UBSan's clean ownership run means those inserted checks did not fire on that path.

A failed compilation prints `SKIP: ASan/UBSan instrumentation unavailable` and keeps `asan-build.log`. A runtime failure is `FAIL-EVIDENCE` with the log retained. Both are report material. A skip means the layer was not observed. The uninstrumented ownership check inside `WEEK11 PASS` is the other layer, the plain binary's assertions, and it remains.

7. ThreadSanitizer instruments conflicting accesses

The TSan section compiles `thread_lab.c` with `-fsanitize=thread -O1` and, if the binary links, runs `thread_lab race` with `TSAN_OPTIONS=exitcode=66`.

The compiler inserts calls on memory accesses and on synchronization operations the runtime understands: thread create and join, mutex operations, atomic operations with their memory orders. The runtime records which thread touched which location and which happens-before

edges it has seen. When it observes a pair of conflicting accesses that those edges do not order, it reports a data race. That is a direct instrument of the Day 2 definition, for the schedules and the synchronization kinds this runtime models.

The wrapper then sorts the outcome, and the sort is the lesson:

```
build failed
  SKIP: TSan instrumentation unavailable

process ran, status 66, and the log contains "data race"
  PASS: ThreadSanitizer detected the intentional race

log matches a fatal runtime / unexpected memory mapping
  SKIP: TSan built but the runtime is unavailable or incompatible

anything else, including a process that exits 0 with no report
  SKIP: no recognized race report (with the status)
  the wrapper's own words: not proof of safety
```

This host may lack `libtsan`. The compiler can also succeed at the link and then fail at process start because the runtime's memory layout does not fit the process, which is the "unexpected memory mapping" branch. A skip on either branch records that the instrumented execution did not produce a race report. It does not convert the racy source into a defined program, and it does not satisfy the deterministic suite. The suite already passed or failed on its own, with TSan out of that recipe. Absence of a report is not proof that the accesses were ordered.

8. One stopped consumer, and the sentence it supports

Picture a GDB stop of `queue_lab` while a consumer sits in `get`. The wrapper does not perform this stop — its breakpoint is `mode_basic` — but the picture is the one you will want the moment a real wait looks stuck.

`info threads` lists the tasks in the thread group. `thread apply all bt` walks each stack. A blocked consumer's frames should include `get`, then `pthread_cond_wait`, then the libc frame that entered the `futex` syscall if the call has crossed into the kernel. The instruction pointer is in libc or in the kernel entry path. The stack pointer addresses that thread's stack, which remains mapped; Day 4's point about the `while` frame surviving the sleep is exactly this stack. Printing `queue.count` reads one stable word because every thread of the process is stopped. A snapshot with `count` 0 and this consumer inside `cond_wait` supports one sentence: at this instant the ring was empty and this waiter was published. It does not say whether a producer is about to signal, whether the wakeup that returns will be spurious, or whether the other consumer will take the next item first. `continue` ends the snapshot.

The registers at the stop are the architectural registers of each thread, the same names as in the assembly lessons. If GDB stopped every thread, no thread of this process is retiring instructions, and the core runs whatever else the scheduler has. If a future session stops only one thread, the others keep running and the snapshot is partial. The wrapper's batch stop is the small case: one extra thread, a join, `info threads`, then `continue` until `pthread_create_join=ok`. Practice the commands there. Use the deadlock lab's `ETIMEDOUT` results when the question is the cycle.

An ASan abort is a different kind of stop. A redzone or use-after-free report means an instrumented load or store touched an address the shadow map calls inaccessible. The access stack is the faulting instruction. The allocation stack, when the runtime prints one, is the `malloc` that owned the neighboring bytes. The ownership mode's wrapper success is the lack of that abort on the instrumented binary, which is a different file from the four SHA-256 lines. Name it as the `-fsanitize=address,undefined` build when you cite it. "No ASan report on ownership" is the whole claim the `PASS` line earns.

A `PASS` on this section means the opposite of a quiet program. It means the tool saw the race the source contains. That is the success condition for an intentional specimen. It is still one instrumented execution. TSan does not promise to report every race in every schedule, and it does not understand synchronization it was not told about. A home-grown spin on a plain `volatile` flag, for example, is not a pthread mutex and is not a C11 atomic release/acquire pair; the runtime may report a race because, in the model, there is one.

The plain `thread_lab race` line earlier in the report and the TSan section are two observations of related source. The first is an uninstrumented run of undefined behavior, useful as a specimen of "a number came out." The second is an instrumented run whose purpose is to name the conflicting accesses. Keep both, and do not let either one's exit code rewrite the other.

9. ToyOS: two kernel tasks, cooperative and preemptive

The course kernel at projects/toyos/src/tasks.c is the first system in the comparison. At the milestone that file implements, `tasks_init` creates two ring-0 tasks with 8192-byte stacks and prints that there are no user processes yet. Each task has a `struct context` holding `r15`, `r14`, `r13`, `r12`, `rbx`, `rbp`, `rsp`, and `rip` — the callee-saved registers plus the instruction pointer `context_switch` exchanges. The states are `TASK_RUNNABLE`, `TASK_RUNNING`, and `TASK_BLOCKED`.

One worker calls `schedule` in a loop. That is the cooperative path: the task gives up the core at a source line it chose. The other worker spins on `timer_ticks` and is switched by `scheduler_tick`, which the timer interrupt calls. That is the preemptive path: the task loses the core between its own instructions. Week 7's task lesson records the transitions that exist: created to runnable to running, running back to runnable on a switch, running to blocked when the task finishes. `TASK_BLOCKED` there means finished. There is no wait channel, no condition predicate, and no wakeup list. `schedule` walks a two-slot array on one CPU and updates state without a lock, which Week 7 notes is viable because the interrupt behavior is tightly constrained.

There is no C memory model in this scheduler. The file is freestanding kernel C. It does not include `<stdatomic.h>`, it does not call `pthread_mutex_lock`, and the two tasks share one kernel address space with private stacks. A data race in the C abstract-machine sense is a hosted, multi-threaded C notion from Days 2 and 5. ToyOS's correctness argument for these tasks is the one Week 7 uses: explicit states, explicit stacks, a context record, and a single core. Later milestones add user address spaces and a small syscall surface. They do not import glibc's pthread condvar or the C11 order parameters into this scheduler. When you compare, compare mechanisms that are actually in the milestone you built.

10. A teaching kernel's sleep, and Linux beside it

The second system is the small teaching OS the wrapper names: pin a revision, then read its `proc` structure, its spinlock, and its `sleep`/`wakeup`. The idea those functions exist to teach, which you should confirm against that pinned source before quoting it, is Day 4's protocol living inside the kernel. The lab's version of that protocol lives in glibc.

```
waiter, holding the spinlock that protects the predicate
  sees the predicate false
  sleep(channel, lock)
      records the channel
      releases the lock
      marks the process asleep
      yields the CPU
waker, holding the same lock
  makes the predicate true
  wakeup(channel)
      marks sleepers on that channel runnable
waiter runs again
  reacquires the lock
  rechecks the predicate
  sleeps again if another thread consumed the event
```

The channel stands in for the condition variable. The spinlock stands in for the mutex. The recheck stands in for the `while`. Lost wakeup is the same bug as Day 4's trace: signal (here, `wakeup`) before the sleeper is published on the channel, with the lock dropped across the gap. The teaching kernel closes the gap by requiring the lock to be held on entry to `sleep` and by releasing it inside `sleep` after the sleeper is recorded. Spurious or stolen wakeups are why the waiter loops. This course does not vendor a particular xv6 tree, so line numbers belong in your report only next to the revision you opened. Week 7's comparison lesson lists the usual landmarks (`proc`, the switch assembly, traps) and asks for that revision explicitly.

The third system is Linux, and it is the one the lab actually runs.

A pthread is a task created so that it shares the process address space, the file-descriptor table, and the signal dispositions, with a private stack, a private register set, and thread-local storage. Day 1 separates that POSIX description from the `clone` flags a given glibc uses. The shared address space is why the queue's `count` is one virtual address for every thread, translated by one set of page tables. The private registers are why a context switch has to save them, which ToyOS does in miniature in `struct context` and Linux does on the way into `schedule` and at syscall entry.

When the lab blocks, it blocks in the kernel on a futex. Day 4 quoted `futex_wait_setup` and `futex_wait` from tree `c9b2385d6793`: the hash-bucket lock is held across the test of the userspace word, a spurious wakeup retries, and the task state store that decides to sleep carries a barrier documented on `set_current_state`. That is Linux's wait path. glibc's `pthread_cond_t` remains userspace; the futex is the syscall it uses when it needs the scheduler. The C memory model remains the rule for the lab's own accesses: the queue is defined because the mutex edges order `count`, the relaxed counter is defined because the object is atomic, and `thread_lab race` is undefined because neither is true there. The kernel's `smp_store_release` mappings from Day 5 are how *this kernel tree* orders its own flag stores on x86-64, AArch64, and RISC-V. They are not a third implementation of `memory_order_release` inside the lab binary.

Put the three side by side only at the level each one actually implements:

```
                   ToyOS kernel tasks     teaching sleep/wakeup     Linux + C lab
-----------------  ---------------------  ------------------------  ------------------------
who runs           2 ring-0 tasks         a process table           tasks, pthread via clone
stacks             private kernel stacks  per-process kernel stack  per-thread user stack
preemption         timer tick and yield   a scheduler you pin       the host scheduler
waiting            BLOCKED = finished     sleep on a channel        futex, condvar in glibc
predicate recheck  none on that path      the sleep loop            while around cond_wait
memory model       freestanding, no C11   the OS's own locks       C11/pthreads for the lab
coherence story    one CPU in that code   depends on the port       the host's caches, Day 6
```

The last row records scope. ToyOS at that milestone does not have a second cache to bounce. A teaching OS ported to several harts inherits that hart's memory model and has to say so. The lab inherits the host, and Day 5's point still stands: an x86-64 litmus count is a fact about the host.

11. Version discipline, written as part of the answer

Every comparison sentence in the report should carry the versions that make it true.

```
lab binaries        the four SHA-256 lines
host kernel         uname -a from the report
excerpt tree        c9b2385d6793, Makefile version 5.10.134
                    paths quoted on Days 4–6 only
C compiler          cc --version, plus -O1/-O2/sanitizer flags per section
C library           the glibc (or other libc) that provided pthread and,
                    if TSan ran, the sanitizer runtime
ToyOS               the course tree's tasks.c at the milestone you built
teaching OS         revision, architecture, and the files you actually opened
tools               gdb and valgrind versions when they ran; the skip text
                    when they did not
```

A sentence such as "Linux waits the way ToyOS waits" fails this test on its face: ToyOS's blocked state in the scheduler milestone is completion, and Linux's futex wait is a published waiter plus a recheck. A sentence such as "the kernel uses `stlr` for release" fails it unless the architecture is AArch64 and the tree is the one whose `arch/arm64/include/asm/barrier.h` you are looking at. On the x86 header of the same tree, store-release is a compiler barrier and a plain store. The report is where those qualifications live, next to the hashes.

Tool absence follows the same rule. A skip is a recorded observation of the environment. The wrapper's TSan skip says the instrumented race was not shown. The race in the source is unchanged. Fill the comparison and the limits section from the source you read and the logs you have. Leave a blank row blank. A stack trace belongs to the binary that produced it, and a line number belongs to the revision you opened.

12. Mastery: Explain · Draw · Observe · Build

Explain

- Which question GDB answers at a stop, which question Memcheck answers about `queue_lab`, and which question TSan answers about `thread_lab race`.

- Why a TSan skip, including a missing `libtsan` or a quiet exit, does not clear the data race, and why a TSan report on the intentional specimen is the section's success.
- Why `WEEK11 PASS` can be true in the same report as an intentional race specimen.
- How ToyOS's `TASK_BLOCKED` differs, in the milestone you can point at, from a teaching-OS `sleep` and from a Linux futex wait.
- Which two kernel identities a careful report names: the running `uname`, and the excerpt tree `c9b2385d6793`.

Draw

Draw the four-layer picture from section 1 and place each tool on the arrow it observes. Draw the three-system table from section 10 with one extra column you fill only from files you opened: a function name for ToyOS (`schedule` or `scheduler_tick`), a function name from the pinned teaching OS, and `futex_wait` from the excerpt tree.

Observe

Run challenges/day-007-debug-report.sh and keep the temporary report path it prints. In your notes, copy the identity block, the `WEEK11 PASS` line, the race line, the deadlock line, every `PASS` / `SKIP` / `FAIL-EVIDENCE` line, and the limits line. If TSan skipped, quote the skip text and write what it establishes about the tool and what it leaves unchanged about the source.

Build

Write the debugging report the wrapper's landmarks section asks for, as prose around those logs. Include the binary hashes, the layered tool results, the three-system comparison at the resolution of section 10, and an explicit list of what this run does not prove: unexecuted schedules, fairness, a kernel you did not boot, and a C behavior inferred from one ISA's litmus count. The report is the deliverable. Completing the wrapper is the evidence underneath it, and the week's mastery is the explanation you can give from that evidence, which remains yours to finish.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Debug Report

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 11-threads-and-concurrency/challenges/day-007-debug-report.sh
```

- View lab source on GitHub